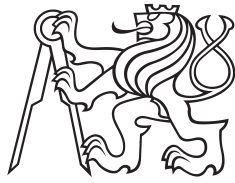


Master Thesis



Czech
Technical
University
in Prague

F3

Faculty of Electrical Engineering

Numerical Optimal Control for Minimum Lap Time

Bc. Richard Ciglanský

Supervisor: doc. Ing. Zdeněk Hurák, Ph.D.
May 2026

Acknowledgements

Děkuji ČVUT, že mi je tak dobrou *alma mater*.

Declaration

Prohlašuji, že jsem předloženou práci vypracoval samostatně, a že jsem uvedl veškerou použitou literaturu.

V Praze, 22. May 2026

Abstract

Thesis researches and implements ways of solving minimum maneuvering time problem (lap time simulation) for road vehicles. Thesis divides this problem into smaller subproblems, for which there are already mature software methods available. Subproblems are: racer car modelling, transcription of optimal control problem into NLP, solving of NLP, solving sets of linear equations and sensitivity analysis. Thesis implements these methods and benchmarks their performance. x

x
x
x
x
x
x
x
x
x
x
x
x
x
x
x

Keywords: word, key

Supervisor: doc. Ing. Zdeněk Hurák, Ph.D.

Abstrakt

Diplomova práca skúma metódy riešenia minimum maneuvering time problemu pre cestne vozidla a implementuje ich. Práca delí tento problém na menšie podproblémy: vytvorenie modelu vozidla, zápis problému optimálneho riadenia, riešenie nelineárneho programu, riešenie systému lineárnych rovníc a citlivosťná analýza. Pre tieto podproblémy existuje mnoho rôznych možností riešenia, práca tieto možnosti vysvetľuje a porovnáva. V práci sú implementované metódy riešenia podproblémov ktoré som považoval za najužitočnejšie. Práca obsahuje aj benchmark vybraných možností riešenia.

Klíčová slova: NLP, laptime simulation, ipopt, skopirovat z bakalarky

Překlad názvu: Metody numerického optimálního řízení pro minimalizaci času na kolo — Cesta do tajů kdovíčeho

Contents

1 Introduction	1		
1.1 Motivation	1		
1.2 Goals	1		
1.3 Structure of thesis	2		
1.4 Literature overview/ survey of laptime simulators	2		
Part I			
Theory			
2 Vehicle and Track modelling framework	7		
2.1 Model like in MSD	7		
2.2 Tire - road interface modelling	7		
2.2.1 Linear tire	8		
2.2.2 Pacejka	8		
2.2.3 Extended Pacejka	9		
2.3 Chassis model	9		
2.4 Wheel Pivot model	9		
2.5 Aerodynamics model	10		
2.6 Suspension model	11		
2.7 Gearbox model	11		
2.8 Accumulator model	12		
2.9 Motor model	12		
2.10 Car models	12		
2.10.1 Generic car model	12		
2.10.2 Mass Point	14		
2.10.3 Single track	14		
2.10.4 Formula Student twin track	15		
2.10.5 Formula Electric twin track	15		
2.11 Track modeling	15		
2.12 Change of independent variable, time to path	15		
3 The Optimal Control Problem	17		
3.1 Solving controlled ODE, initial vlaue problems and boundary value problems	17		
3.1.1 Initial vlaue problem	18		
3.1.2 Forward backward pass	18		
3.2 Transcription methods	18		
3.2.1 Shooting methods	18		
3.2.2 Nonlinear Programming with Runge Kutta constraints	19		
3.2.3 Collocation methods	20		
3.3 Gauss methods as pseudospectral collocation	21		
3.4 Gauss methods as h-method	22		
		3.5 Diving problem into many lower degree polynomials	22
		3.6 Mesh refinement methods	22
		3.7 Mesh refinement	22
		3.7.1 collocation integral form	22
		3.7.2 collocation differential form	22
		3.7.3 p adaptive methods	22
		3.7.4 h adaptive methods	23
		3.7.5 hp adaptive methods	25
		3.8 Adjoint methods	25
		3.8.1 quadrature rules	25
		4 Solvers for nonlinear programming	27
		4.1 Nonlinear programming	28
		4.1.1 KKT conditions	28
		4.2 Interior point methods: IPOPT, MadNLP	29
		4.2.1 Barrier problem formulation	29
		4.3 Sequential quadratic programming	30
		4.4 Calculation of derivatives	30
		4.5 Scaling and matrix conditioning	30
		4.6 Sparsity	31
		4.7 Initialization	31
		4.8 Global and local optimums	31
Part II			
Implementation of Optimal Control problem			
		5 Frameworks for Optimal Control Problem Formulation	35
		5.1 GPOPS-II	36
		5.2 CasADi	36
		5.3 dymos + OpenMDAO	36
		5.4 Acados	36
		5.5 Julia + JuMP/ infititeopt/ examomdels	36
		6 My implementation, julia + JuMP	39
		6.1 sensitivity Analysis	40
Part III			

	results	
7	comparison of methods	45
8	Conclusion	51
	Appendices	
A	Bibliography	55

Figures

2.1 Wheel Pivot component: velocity inputs (top-left) transform to tire frame (top-right), tire forces (bottom-right) transform to forces and moments on CoG (bottom-left).....	8	2.8 Generic car model structure. Control and state vectors are mapped to car parameters, which are then used by the parametrizable car ODE to compute derivatives.....	13
2.2 Tire component: tire-frame velocity $\mathbf{v}_{tire}$ and drive torque T_{in} on the left, normal force F_z from suspension on top, output is the tire force vector $\mathbf{F}_{tire}$ consumed by the wheel pivot.....	8	2.9 Single-track (bicycle) model computational graph. Velocity flows from CoG through wheel assemblies to tires, forces flow back. Motors provide torque through gearboxes, aerodynamics provides drag and normal forces.	14
2.3 Wheel Pivot inputs and outputs. Steering angle δ_i is a control input. CoG velocity $\mathbf{v}$ and angular velocity $\boldsymbol{\omega}$ are mapped to tire-frame velocity $\mathbf{v}_{t_i}$, which is consumed by the tire model. Tire force vector $\mathbf{F}_{tire}$ is rotated and translated into forces $\mathbf{F}_{CoG}$ and moment $\mathbf{M}_{CoG}$ acting on the chassis.	10	3.1 Flowchart of transcription method classification, adapted from [40]	18
2.4 Aerodynamics component: longitudinal velocity v_x maps to drag force F_{drag} and downforce F_{down}	10	3.2 Transcription layer: choice of orthogonal polynomial basis (top row), the resulting collocation point schemes derived from the Legendre basis (middle row), and the two equivalent algebraic formulations available for each scheme — integral and differential form [23] (bottom row). Vertical dots under the other bases indicate that they admit their own families of quadrature schemes, which are not expanded here. . .	20
2.5 Simple suspension component: downforce F_{down} , center of pressure CoP and chassis geometry (mass m , CoG offsets x_{CoG}, y_{CoG}) map to the four wheel normal loads $F_{z,FL}, F_{z,FR}, F_{z,RL}, F_{z,RR}$	11	3.3 Mesh refinement layer: the four standard strategies for refining a collocation mesh. p - and h -refinement vary only the polynomial degree or only the segment count, respectively; hp - and ph -refinement combine both, differing in which decision is made first within each refinement iteration [20, 48].....	22
2.6 Gearbox component: motor-side torque T_{in} enters from the left, wheel-side angular velocity ω_{out} enters from the right; outputs are the geared-up torque T_{out} to the wheel and motor-side angular velocity ω_{in} to the motor.	11	3.4 Node-interval ODE error for power-test case.....	23
2.7 Motor component: electrical power P_{elec} enters from the battery side, shaft angular velocity ω is imposed by the drivetrain, output is shaft torque T	12	3.5 Node-interval ODE error for power-test case.....	24
		3.6 Node-interval ODE error for power-test case.....	24

4.1	Unifying framework: the “wheel of strategies”[57]	27
4.2	NLP solution flow: problem statement → solver family (SQP / IPM) → derivative computation → assembled Newton step.	28
5.1	Optimal control problem solving pipeline.	35
6.1	Optimization software stack . . .	39
6.2	Node-interval ODE error for power-test case.	40
6.3	NLP solution flow with the methods used in this thesis highlighted in green: interior point with barrier reformulation (IPOPT, MadNLP), automatic differentiation provided by JuMP, and the quasi-Newton (L-BFGS) Hessian approximation used in IPOPT’s limited-memory mode.	41
6.4	Mesh refinement layer with the strategies implemented in this thesis highlighted in green: p -refinement (raising the polynomial degree on fixed segments) and h -refinement (subdividing segments at fixed degree).	41
6.5	Transcription layer with the methods used in this thesis highlighted in green: Legendre basis with all three Gauss-type quadrature schemes (LG, LGR, LGL), where LG and LGR are implemented in differential form and LGL in integral form.	42

Tables

2.1	Parameters of the aerodynamic model.	10
2.2	Parameters of the gearbox model.	11
2.3	Parameters of the motor model.	12
5.1	Comparison of Julia (JuMP + InfiniteOpt) and Python (OpenMDAO + dymos) stacks for optimal control problem formulation.	38
7.2	Benchmark summary statistics by transcription variant.	45
7.4	Benchmark results by track, vehicle model, and transcription variant.	46
7.6	Linear solver benchmark summary statistics.	46
7.8	Linear solver benchmark results by track, vehicle model, and transcription variant.	47
7.10	NLP solver / backend benchmark summary statistics (IPOPT-MUMPS, MadNLP-CPU, MadNLP-GPU).	48
7.12	NLP solver / backend benchmark results by track, vehicle model, and transcription variant.	49
7.14	Hessian approximation benchmark summary statistics (exact vs. limited-memory quasi-Newton).	49
7.16	Hessian approximation benchmark results by track, vehicle model, and transcription variant.	50

Chapter 1

Introduction

1.1 Motivation

Topic of this thesis is motivated, by my work in Formula Student team eForce Prague Formula, where I worked on real-time vehicle dynamics control software. There I got intricately exposed to problems faced by racing teams, such as should we increase the aerodynamic downforce at the cost of higher drag? Should we put accumulator in the middle of vehicle, to decrease moment of inertia? Should the aerodynamics be designed for straight line drive, or for steady state turning? How to use limited energy from accumulator more effectively, how faster can we go if we made better power limiting algorithm? How large the accumulator should be? Which torque allocation algorithm should be used? How to design the suspension kinematics to maximize the tire utilization? Is rear wheel steering worth the increased weight? All of these problems can be analyzed by using purpose built tool, good engineering practices or using pen and paper with heavily simplified model. Formula student season is 10 months for development and construction of the vehicle, 2 months for competition. That puts enormous pressure on deciding on vehicle concept fast. In my opinion what formula student teams need is a tool to quickly evaluate different car concepts. As my bachelor thesis I created a prototype of such tool, goals of this thesis come from computational and user experience from my previous tool.

1.2 Goals

- Primary goal of this thesis is to investigate available methods for numerical optimal control with respect to their suitability for planning an optimal trajectory of a vehicle aiming to achieve the minimum lap time on a racing circuit. To do this for various vehicle concepts, thesis has to present a vehicle modelling framework.
- Benchmark performance of available numerical methods
- Thesis should present a tool for convenient vehicle modelling, and vehicle concept evaluation.

- Support vehicle parameter optimization, using any vehicle parameter as control input and sensitivity analysis of vehicle parameters.
- Goal is not precise vehicle modeling and validation, goal is to create a framework to perform vehicle concept analyses easily.

1.3 Structure of thesis

First part of thesis with vehicle modelling in the context of optimal control. A universal model structure inspired by [16] is presented. Then the theory of optimal control is presented.

1. ODE models of cars
2. Solving methods: time marching forward, backward pass, NLP,
3. Collocation (p, h, hp refinement), integral + differential form (Gauss-Radau, Legendre, Lobatto)
4. Implementation overview, Algorithmic differentiation, JUMP/Examples/InfiniteOpt + Julia, Python/MATLAB, CasADi, interior point solvers
5. Actual implementation and results
6. Conclusion

1.4 Literature overview/ survey of laptime simulators

Early attempts at lap-time computation started in 1950's. In 1970s computers became commonly used to solve MLTS using Quasi Steady State methods (QSS), these methods are what most race engineers think of when laptime simulation is mentioned. Here a car is modeled as point mass moving along predefined trajectory with lateral and longitudinal accelerations limited by steady state performance envelope[52], transient dynamics such as yaw acceleration are neglected. Formulations of MLTS as OCP appeared in late 1980s [28]. Article which summarizes history and state of the art methods of computing MLTS is [44]. Article [50] is widely regarded as the reference work, many parts of this thesis are taken from it. This article defines the track model, change of independent variable from time to distance traveled, and a twin-track Formula 1 model, with some parameters as decision variables. It is a very good description of most modern MLTS solving methods using NLP. Methods of computing minimum lap time is still active topic in race engineering, as for determining better setup of a race car for a given track, on a given day yields significant advantage. With more precise model the accuracy of setup improves, but also the computational time increases. In [2] authors deal with computational efficiency of computing minimum lap-time,

by using sequential convex programming. SEM SKOPIROVAT VECI Z
BAKALRKY ZO STATE OF THE ART OVERVIEW

A very popular laptime simulator is OptimumLap from OptimumG [3], It is a free commercial tool advertised to achieve 10% accuracy with real vehicles. It uses Quasi Steady State methods (QSS) This is the standard tool for laptime simulations. Open source implementation of the same method [45], My implementation of this method in chapter ??, QSS methods are very fast, but the models for which they can be used is equally very simple.

Popular open source C++ implementation of laptime simulator using nonlinear programming and automatic differentiaoton[43]. It uses IPOPT and CppAD[7]. Vehicle model is 6DOF twintrack. Other implementations are work from austria or germany[29] class project from Georgia institute of technology [1]



Part I

Theory

Chapter 2

Vehicle and Track modelling framework

In assignment of this is

A key aspect will be the selection of a sufficiently accurate yet computationally tractable model of the vehicle

There is not one model of vehicle that would fit all requirements of race engineers. For each problem, a different complexity of the vehicle model is sufficient, e.g. for analysis of power limit algorithm, the lateral dynamics of the vehicle can be heavily simplified and simple transcription and solving methods yield sufficient accuracy within a fraction of a second, this is described in ???. Whereas analysis of torque allocation requires the full complex model and more computationally expensive transcription and solving methods. Therefore, a vehicle modeling framework is needed to satisfy the needs of different types of analyses.

2.1 Model like in MSD

The structure of the vehicle model has to allow extending/changing components, so that different vehicle concepts can be simulated, with minimal coding. Book in [16] describes modelling framework that allows to create car components as modules with flow and effort interface. This common interface then heavily simplifies process of creating a car model, as modules can be reused and chained together easily. ADD DESCRIPTION OF THE BROWN GRAPH THEORY HERE. velocity goes one direction and forces go the other direction, must give better description.

Vehicle components are chassis, wheel pivot, tire, aerodynamics, suspension, motor

2.2 Tire - road interface modelling

Job of the tire model in this thesis is to calculate forces acting on tire from the relative velocity and angle between tire and road. Input of general tire model is 3 speeds and 3 angles, output is 3 forces.

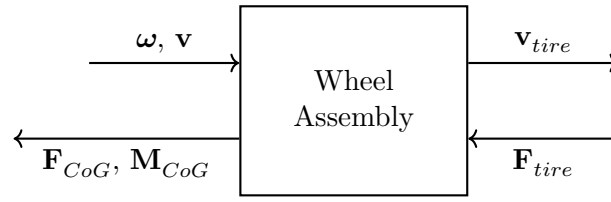


Figure 2.1: Wheel Pivot component: velocity inputs (top-left) transform to tire frame (top-right), tire forces (bottom-right) transform to forces and moments on CoG (bottom-left).

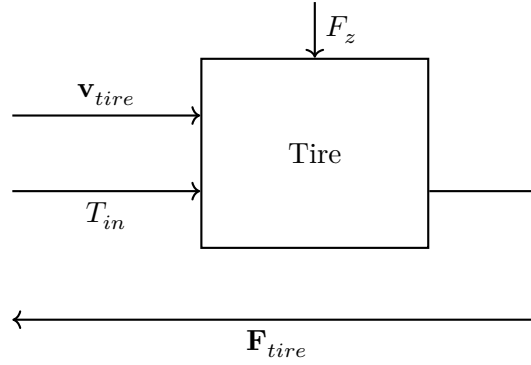


Figure 2.2: Tire component: tire-frame velocity $\mathbf{v}_{tire}$ and drive torque T_{in} on the left, normal force F_z from suspension on top, output is the tire force vector $\mathbf{F}_{tire}$ consumed by the wheel pivot.

Most forces that govern vehicle movement are transferred by the tires. The tire is considered to be the most important system of a vehicle and it is notoriously difficult to model [46]. Thus tires need to be modeled with great caution in mind. Not only are there many parameters, such as tire wear, tire temperature, road roughness, road wetness, tire pressure, etc., that affect the resultant forces, they are also constantly changing during the lap. Therefore they are difficult to measure and model precisely [18].

2.2.1 Linear tire

Simplest tire model. Tire is defined by friction ellipse and an angle at which the slip angle is highest

2.2.2 Pacejka

Pacejka tire uses

2.2.3 Extended Pacejka

2.3 Chassis model

point mass with definition of wheel pivot points.

$$\mathbf{M}_{CoG,i} = \mathbf{r}_i \times \mathbf{F}_i, \quad \mathbf{F}_i = \begin{bmatrix} F_{x_i} \\ F_{y_i} \\ 0 \end{bmatrix}. \quad (2.1)$$

$$\mathbf{F}_{CoG} = \sum_i \mathbf{F}_i + \begin{bmatrix} F_{\text{drag}} \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{M}_{CoG} = \sum_i \mathbf{M}_{CoG,i}. \quad (2.2)$$

$$\dot{\mathbf{v}} = \frac{\mathbf{F}_{CoG}}{m} - \boldsymbol{\omega} \times \mathbf{v}, \quad \dot{\boldsymbol{\omega}} = \frac{\mathbf{M}_{CoG}}{I}. \quad (2.3)$$

$$\mathbf{x} = \begin{bmatrix} v_x \\ v_y \\ \psi \\ \omega_z \end{bmatrix}, \quad \dot{\mathbf{x}} = \begin{bmatrix} \dot{v}_x \\ \dot{v}_y \\ \dot{\omega}_z \end{bmatrix}. \quad (2.4)$$

2.4 Wheel Pivot model

Wheel pivot point is modelled as transformer. It defines the transformation of CoG velocities into tire frame and then transforms tire forces and torque into CoG frame. Angular velocity and position vectors are formed from yaw rate $\dot{\psi}$ and the longitudinal distance l_i of the axle from the center of gravity:

$$\boldsymbol{\omega} = \begin{bmatrix} 0 \\ 0 \\ \dot{\psi} \end{bmatrix}, \quad \mathbf{r}_i = \begin{bmatrix} l_i \\ b/2 \\ 0 \end{bmatrix}, \quad (2.5)$$

where b is the track width. Velocity at each tire pivot point is:

$$\mathbf{v}_{p_i} = \mathbf{v} + \boldsymbol{\omega} \times \mathbf{r}_i. \quad (2.6)$$

Velocities are then rotated from the pivot frame into the tire frame using the steering angle δ_i :

$$R_t = \begin{bmatrix} \cos \delta_i & -\sin \delta_i & 0 \\ \sin \delta_i & \cos \delta_i & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{v}_{t_i} = R_t \mathbf{v}_{p_i}. \quad (2.7)$$

Tire forces F_{tx} and F_{ty} are computed from $\mathbf{v}_{t_i}$ by the tire model. Forces are then rotated back to the pivot frame:

$$\begin{bmatrix} F_{x_i} \\ F_{y_i} \end{bmatrix} = \begin{bmatrix} \cos \delta_i & \sin \delta_i \\ -\sin \delta_i & \cos \delta_i \end{bmatrix} \begin{bmatrix} F_{tx} \\ F_{ty} \end{bmatrix}. \quad (2.8)$$

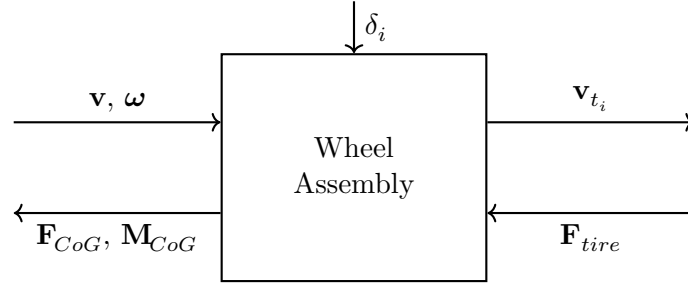


Figure 2.3: Wheel Pivot inputs and outputs. Steering angle δ_i is a control input. CoG velocity $\mathbf{v}$ and angular velocity $\boldsymbol{\omega}$ are mapped to tire-frame velocity $\mathbf{v}_{t_i}$, which is consumed by the tire model. Tire force vector $\mathbf{F}_{tire}$ is rotated and translated into forces $\mathbf{F}_{CoG}$ and moment $\mathbf{M}_{CoG}$ acting on the chassis.

2.5 Aerodynamics model

Aerodynamic model is modelled as nonlinear resistance element, mapping velocity into forces. Drag and downforce scale with the square of longitudinal velocity:

$$F_{\text{drag}} = \frac{1}{2} \rho C_D v_x^2, \quad F_{\text{lift}} = \frac{1}{2} \rho C_L v_x^2. \quad (2.9)$$

The downforce is split between front and rear axles by the center of pressure ratio $\text{CoP} \in [0, 1]$, where $\text{CoP} = 0.5$ corresponds to a balanced distribution. Air density ρ is taken as 1.225 kg/m^3 at sea level. Coefficients C_L , C_D and CoP used per vehicle model are listed in Table 2.1.

Table 2.1: Parameters of the aerodynamic model.

Symbol	Unit	Description
C_L	—	Lift coefficient (negative = downforce)
C_D	—	Drag coefficient
CoP	—	Center of pressure (front axle share, 0–1)
ρ	kg/m^3	Air density

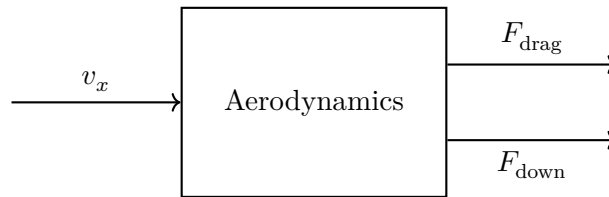


Figure 2.4: Aerodynamics component: longitudinal velocity v_x maps to drag force F_{drag} and downforce F_{down} .

This model is sufficient when vehicle is driving relatively straight, that means lateral acceleration is roughly zero. With high lateral acceleration this model is no longer valid, because air speeds are different on left and right part of car and also the car starts rolling- one side of vehicle is closer than the other, creating imbalance of forces.

2.6 Suspension model

Job of suspension in car is to maintain tire contact, distribute the load from lateral load transfer to better utilize tires and use kinematics to make use of camber thrust. Commonly suspension of car consists of spring, damper and the mounting with tie rods. In this thesis Suspension model had to be significantly simplified. Modelling each wheel with a damper, spring and its own kinematics is beyond the scope of this thesis. However, the structure of allows it, i just didnt have enough time to implement it. this quarter car model would be very beneficial toward the goal of scalability, now without it the model scalability is limited as divigin force between more than two wheel on axis is staticky nedoručeny problem.

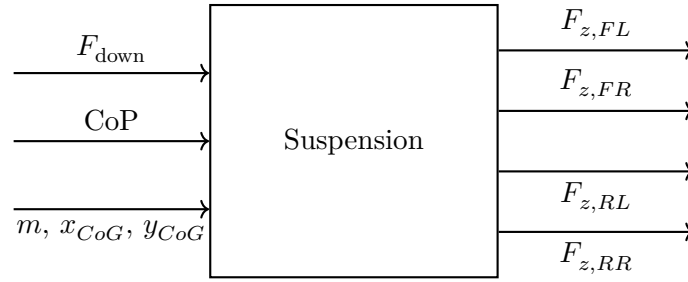


Figure 2.5: Simple suspension component: downforce F_{down} , center of pressure CoP and chassis geometry (mass m , CoG offsets $x_{\text{CoG}}, y_{\text{CoG}}$) map to the four wheel normal loads $F_{z,FL}, F_{z,FR}, F_{z,RL}, F_{z,RR}$.

2.7 Gearbox model

gearbox is the textbook example of transformer and resistive element.

$$T_{\text{out}} = \eta i T_{\text{in}}, \quad \omega_{\text{out}} = \frac{\omega_{\text{in}}}{i}. \quad (2.10)$$

Table 2.2: Parameters of the gearbox model.

Symbol	Unit	Description
i	—	Gear ratio (input to output speed)
η	—	Mechanical efficiency

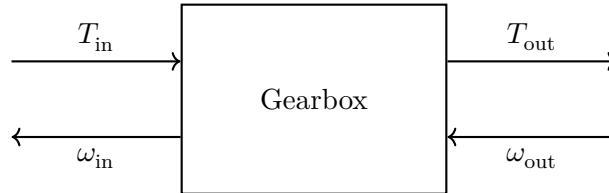


Figure 2.6: Gearbox component: motor-side torque T_{in} enters from the left, wheel-side angular velocity ω_{out} enters from the right; outputs are the geared-up torque T_{out} to the wheel and motor-side angular velocity ω_{in} to the motor.

2.8 Accumulator model

Accumulator is modelled as capacitor from brown, with additional constraints on maximum and minimum power

2.9 Motor model

$$P_{\text{mech}} = T \omega, \quad P_{\text{elec}} = \max\left(\frac{P_{\text{mech}}}{\eta}, P_{\text{mech}} \eta\right). \quad (2.11)$$

Table 2.3: Parameters of the motor model.

Symbol	Unit	Description
T	Nm	Shaft torque
η	—	Motor + inverter efficiency
P_{mech}	W	Mechanical shaft power
P_{elec}	W	Electrical port power (battery side)

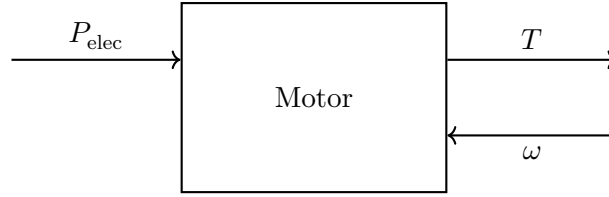


Figure 2.7: Motor component: electrical power P_{elec} enters from the battery side, shaft angular velocity ω is imposed by the drivetrain, output is shaft torque T .

2.10 Car models

These models must work when used in NLP transcription for solving boundary value problem, but they also must work for initial value problem, when the model is initialized or verified for transcription error. Some toolboxes can automatically create constraints from ODE models[61]. In my implementation I have and if statement that switches between IVP and BVP variant of models equations, major disadvantage of this approach is that the equations for both variants must be effectively the same and finding errors is difficult in this.

2.10.1 Generic car model

one of the goals is calability, therefore any car Parameter can be assigned as state, control, static parameter, decision variable or sensitivity analysis variable. This allows for simple RELAXATION? rozvolnenie? of DOF allowing to decide on one line of code which parameters should be controlled. This also allows for finding the optimal parameters of vehicle, such as brake bias,

aerodynamics balance, gear ratio. When parameter is chosen to be sensitivity analysis variable, impact of changing this variable is cheaply computed after the optimisation is finished, see Section 3.8.

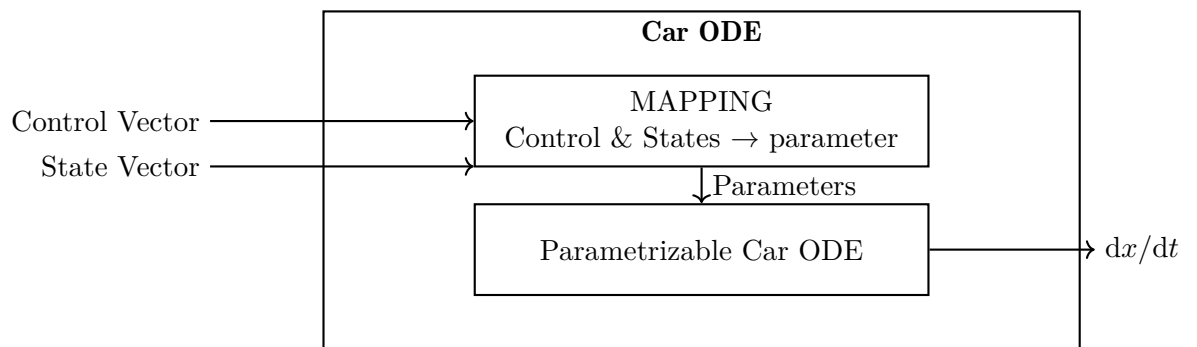


Figure 2.8: Generic car model structure. Control and state vectors are mapped to car parameters, which are then used by the parametrizable car ODE to compute derivatives.

2.10.2 Mass Point

2.10.3 Single track

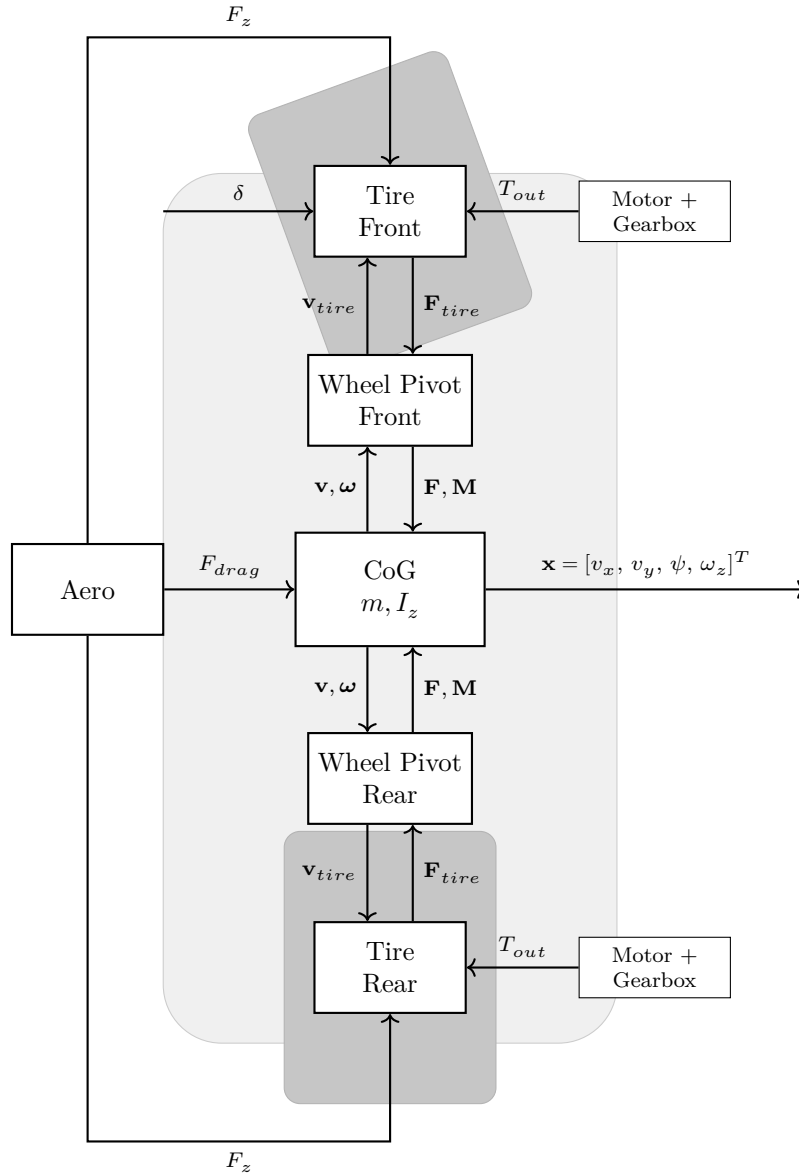


Figure 2.9: Single-track (bicycle) model computational graph. Velocity flows from CoG through wheel assemblies to tires, forces flow back. Motors provide torque through gearboxes, aerodynamics provides drag and normal forces.

2.10.4 Formula Student twin track

2.10.5 Formula Electric twin track

2.11 Track modeling

Track is sampled at discrete arc length nodes s_i and stored as a set of fields evaluated at each node:

- centerline coordinates $x(s)$, $y(s)$,
- heading $\theta(s)$,
- curvature $\kappa(s)$,
- left and right track widths $w_L(s)$, $w_R(s)$,
- air density $\rho(s)$,
- friction coefficient $\mu(s)$,
- inclination $\phi(s)$.

Each field is interpolated by a cubic spline in the arc length s , yielding C^2 -continuous fields along the entire trajectory, no nono no this did not work well, loinear interpolation was better. Obtaining suitable track for optimization from GPS is non-trivial, simple methods lead to noisy data, which then corrupts the optimization result. Therefore smoothing method described in [50] has been used.

2.12 Change of independent variable, time to path

When differential equations for vehicle movement are stated. For use in lap time simulation it is benefecial to describe the car states not in time but in distance traveled. This introduces connection with location on track and reduces number of state variables by one. This tranformation is described in[50].

$$\varepsilon = \psi - \theta(s) \quad (2.12)$$

$$S_f = \frac{dt}{ds} = \frac{1 - n C(s)}{v_x \cos \varepsilon - v_y \sin \varepsilon} \quad (2.13)$$

$$\frac{dn}{dt} = v_x \sin \varepsilon + v_y \cos \varepsilon \quad (2.14)$$

$$\frac{d}{ds} \begin{pmatrix} v_x \\ v_y \\ \psi \\ \omega \\ n \\ t \end{pmatrix} = S_f \begin{pmatrix} \dot{v}_x \\ \dot{v}_y \\ \dot{\psi} \\ \dot{\omega} \\ \dot{n} \\ 1 \end{pmatrix} \quad (2.15)$$

Chapter 3

The Optimal Control Problem

In previous chapter I have described how to build differential equations that govern movement of vehicle. Methods discussed in this section are doing essentially the same thing, solving controlled ODE along a time/path trajectory. But their approach and results are different.

3.1 Solving controlled ODE, initial value problems and boundary value problems

USE BOUNDARY AND INITIAL VALUE PROBLEM INSTEAD OF TIME MARCHING
The most common method for solving controlled ODE is time marching. Where the state computation is always for the states at time $t+1$, after solving at $t+1$, time is moved/marched forward and solution at $t+2$ is calculated and so on. This gives a solution to ODE. If the ODE can be controlled like the car, you also need a separate controller, which becomes part of the ODE. This approach gives a solution to the problem of how to control car, so that it completes a lap around a circuit. However, this solution is not optimal to time or any objective as the controls depend on the controller. Therefore, if the car parameters change and the controller doesn't, the resulting time can be exactly same, because controller cannot take advantage of lighter car. Sensitivity analysis and lap time can be obtained but it is skewed by the controller chosen. In Car maker the controller is driver and they have Driver adaptation. Using time marching is not best for sensitivity analysis and lap time simulation. Controller can be tuned after each change of model but, that is time intensive. Another approach to solving controlled ODE is using shooting methods. Now the algorithm doesn't compute the states at only next time point, but it computes all states at all time points at the same time. Using this method it is possible to get rid of the controller, and instead of control rule a feed forward control vector along the trajectory is computed. This chapter deals with transcribing the continuous problem into discrete steps

3.1.1 Initial value problem

3.1.2 Forward backward pass

DESCRIBE POPULAR LAPTIME SIMULATION METHOD THAT USES FORWARD AND BACKWARD PASS FOR A MODEL OF LOW COMPLEXITY

3.2 Transcription methods

Transcription methods rewrite continuous (infinite-dimensional) OCP into finite-dimensional NLP. That means representing the continuous trajectory of a state by values sampled at discrete points. The terminology of transcription methods can be difficult to understand, as it is complex topic and often articles present conflicting information [40]. Website [40] and other work from Matthew Kelly [37, 35, 38] offers a very digestible introduction into the topic. This thesis focuses only on direct methods. Classification of methods in this thesis is based on [40] Figure 3.1 contrasts the three axes along which transcription methods are commonly classified.

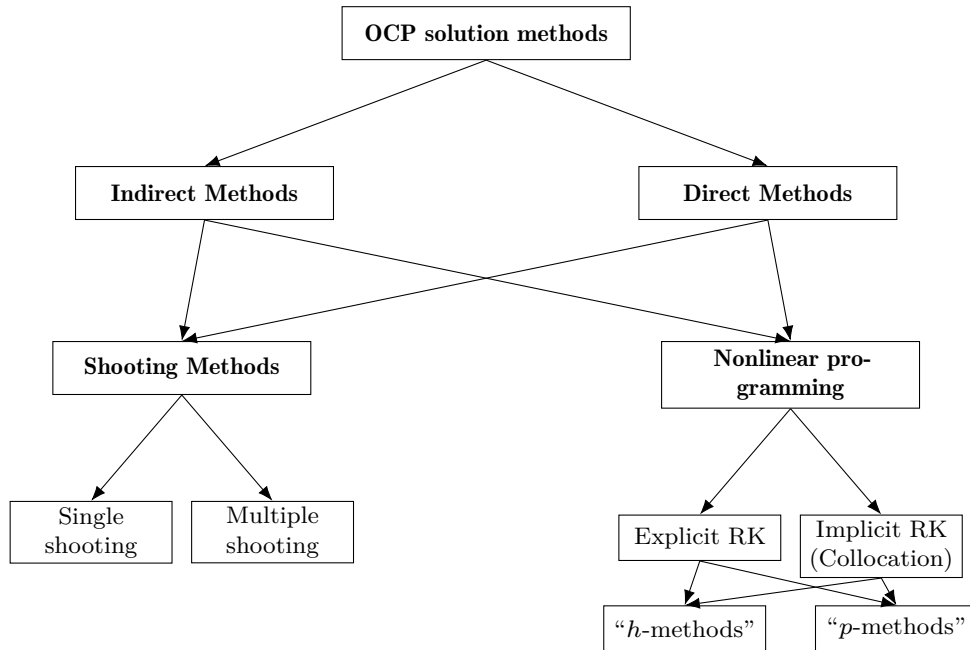


Figure 3.1: Flowchart of transcription method classification, adapted from [40]

3.2.1 Shooting methods

Shooting methods get their name from aiming artillery canon analogy, with given powder load canon and target location find the aiming angle to hit the target. In depth explanation with animation and example code is here [36] This is classical boundary value problem. Methods shoot, miss, adjust aim,

shoot again and iterate until target is hit. These methods are solving boundary value problem using an IVP solver. More precisely,

1. Guess the controls
2. Compute the solution using an initial value problem solver
3. Compare the simulated value at the end of interval from 2. with the boundary value(targets location)
4. Based on error update the controls and go to 2.

Shooting methods are split into single and multiple shooting. They differ in way they handle state discretisation. Single shooting discretizes the state only at start and end of interval and computes the trajectory in one shot. Whereas multiple shooting discretizes the states along the trajectory and computes multiple shots, this can be also parallelized on a CPU [22]. Shooting methods are progressively being displaced by collocation methods [51]

3.2.2 Nonlinear Programming with Runge Kutta constraints

These methods are not using initial value solvers to solve boundary value problem, instead they use nonlinear programming. Most straightforward method to transcribe continuous optimal control problem into non-linear program, is to use explicit Runge Kutta solving methods from initial value problem as equality constraints on the dynamics of a vehicle. Using Euler method, the constraints for the dynamics of vehicle are:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + h \cdot f(\mathbf{x}_k, \mathbf{u}_k), \quad (3.1)$$

where f denotes the vehicle dynamics, h the step size, and $\mathbf{x}$ the state vector. This works, but it may be the worst possible transcription method, due to high error and instability of euler method [4]. Problems of euler method encountered when solving initial value problems also translate to nonlinear programming. Implicit RK methods can be used too, In Initial value problems they are used less often, as they require iterative solving. Optimal control problem is Boundary value problem and not initial value problem, therefore:

...for a boundary value problem ...any scheme becomes effectively implicit. Thus, the distinction between explicit and implicit initial value schemes becomes less important in the BVP context.

[12, p. 69]

For boundary value problem it makes no sense to use explicit Runge Kutta methods, as they have inferior stability compared to implicit Runge Kutta methods.[51, p. 4]. Implicit Runge Kutta methods, which are commonly used are from LobattoIIIA family, but any other method can be used. Book [15] uses only these.

3.2.3 Collocation methods

The objective of collocation method is to find a polynomial that approximates the solution of a boundary value problem along independent variable (most commonly time, in this thesis it is distance travelled along centerline). Collocation methods are a subset of implicit Runge Kutta methods. The main distinction between Runge Kutta methods and collocation, is that the result of collocation is always polynomial, that does not hold for general Runge Kutta method.

Collocation methods are based on theory of orthogonal polynomials

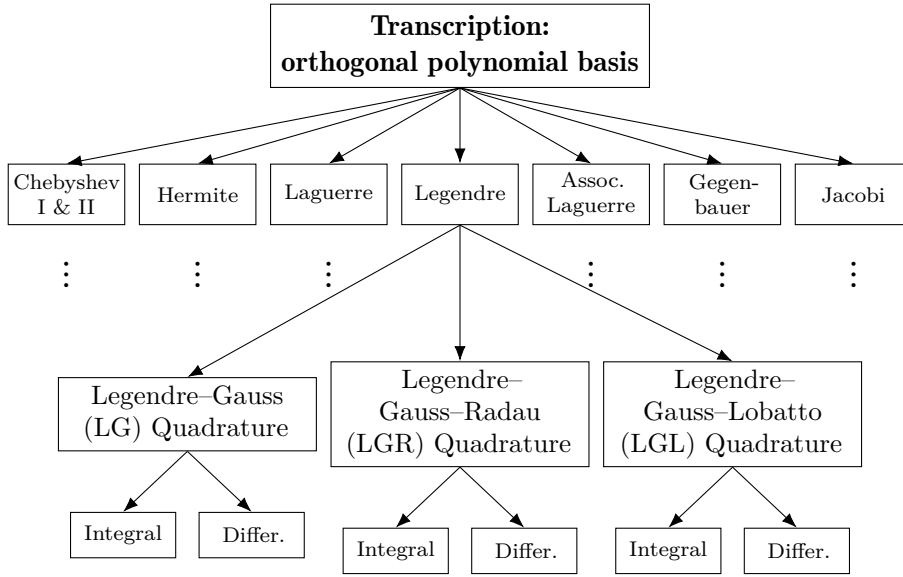


Figure 3.2: Transcription layer: choice of orthogonal polynomial basis (top row), the resulting collocation point schemes derived from the Legendre basis (middle row), and the two equivalent algebraic formulations available for each scheme — integral and differential form [23] (bottom row). Vertical dots under the other bases indicate that they admit their own families of quadrature schemes, which are not expanded here.

The concept of collocation is old and universal in numerical analysis ...For ordinary differential equations it consists in searching for a polynomial of degree s whose derivative coincides ("co-locates") at s given points with the vectorfield of the differential equation ...[5, p. 224]

Collocation methods are the core of this thesis, I have implemented Gauss radau, gauss, gauss lobatto. Collocation methods differ in points at which the derivative of polynomial coincides ("co-locates") with the vector field of differential equation. Commonly used methods in optimal control are radau, lobatto, legendre. They can be written in integral and differential form[23]. Collocation methods in this thesis are based on Framework described in[24]

Polynomial that approximates the solution of ODE is based on Legendre polynomials. Important note that in actual implementation this calculation of

polynomial basis is impractical, therefore barycentric form from [13] has been used as implementation of these methods.

$$L_i(\tau) = \prod_{\substack{j=0 \\ j \neq i}}^N \frac{\tau - \tau_j}{\tau_i - \tau_j}, \quad i = 0, \dots, N \quad (3.2)$$

$$x_j^N(\tau) = \sum_{i=0}^N x_{ij} L_i(\tau) \quad (3.3)$$

the differentiation matrix is made also using barycentric form from [13, p.513] Many articles omit, that this barycentric form is crucial for fast code.

$$\dot{x}_j^N(\tau_k) = \sum_{i=0}^N x_{ij} \dot{L}_i(\tau_k) = \sum_{i=0}^N D_{ki} x_{ij}, \quad D_{ki} = \dot{L}_i(\tau_k) \quad (3.4)$$

then the output?? of differential equation can be aproximated using the differentiation matrix.

$$\dot{x}_j^N(\tau_i) = (D\mathbf{X})_{ij} \quad (3.5)$$

This is then used as constraints and voila nonlinear optimal control problem is born. 😊

$$\begin{aligned} \min \quad & \Phi(\mathbf{X}_{N+1}) \\ \text{s.t.} \quad & D\mathbf{X} = F(\mathbf{X}_{\text{LG}}, \mathbf{U}_{\text{LG}}) \\ & \mathbf{X}_{N+1} = \mathbf{X}_0 + \mathbf{w}^T F(\mathbf{X}_{\text{LG}}, \mathbf{U}_{\text{LG}}) \\ & \mathbf{X}_0 = \mathbf{x}_0 \end{aligned} \quad (3.6)$$

3.3 Gauss methods as pseudospectral collocation

I have implemented Gauss-Legendre, Gauss-Lobatto, Gauss-Radau as global collocation methods, that is one polynomial approximating the entire trajectory. I have implemented them according to [23]

3.4 Gauss methods as h-method

3.5 Diving problem into many lower degree polynomials

3.6 Mesh refinement methods

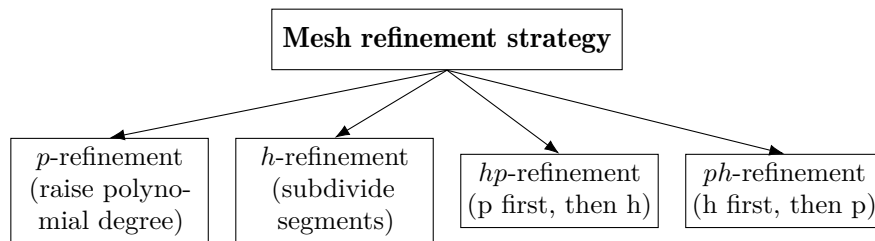


Figure 3.3: Mesh refinement layer: the four standard strategies for refining a collocation mesh. p - and h -refinement vary only the polynomial degree or only the segment count, respectively; hp - and ph -refinement combine both, differing in which decision is made first within each refinement iteration [20, 48].

3.7 Mesh refinement

3.7.1 collocation integral form

integral form is straight forward, same as other unge kutta Methods

3.7.2 collocation differential form

differeanital form requires to create differentiation matrix of the aproximating polnomial,which contains the difference of the approximating polynomial.

3.7.3 p adaptive methods

p adaptive methods work by increasing the order of polynomial, until the error is below user defined threshold. Figure ?? shows that error along track is not distributed equally. Error is localised around areas with large jump in controls. These areas need finer sampling. P methods cannot focus these specific points directly, they focus entire trajectory.

The errors are concentrated around the discontinuity of controls, where the behaviour of the vehicle is more complex/ requiring high order or finer approximation. Figure shows this??

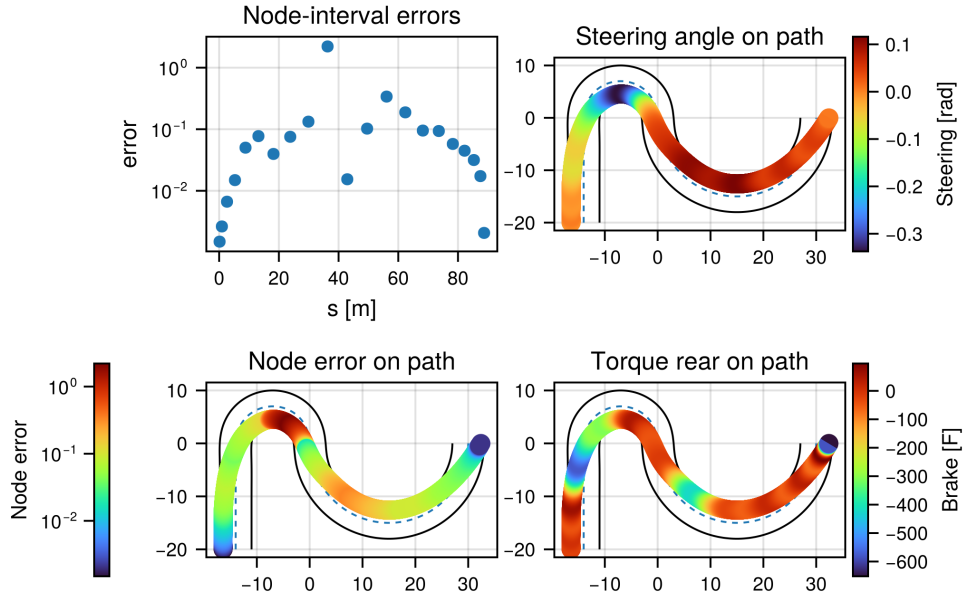


Figure 3.4: Node-interval ODE error for power-test case.

3.7.4 h adaptive methods

h methods are used when solution of optimal control problem is approximated by many intervals with low degree polynomial. These methods can decide to split segment into two smaller segments, they use different techniques to do this. Method described in [15] calculates error using DOPLNIT. Another method described in [31] uses techniques from essentially nonoscillatory (ENO) interpolation

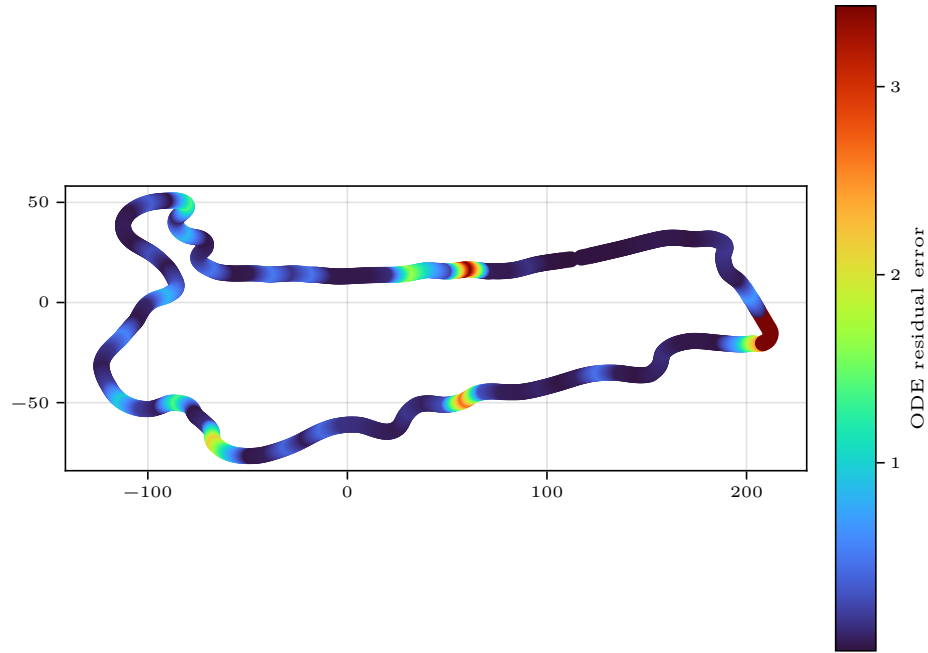


Figure 3.5: Node-interval ODE error for power-test case.

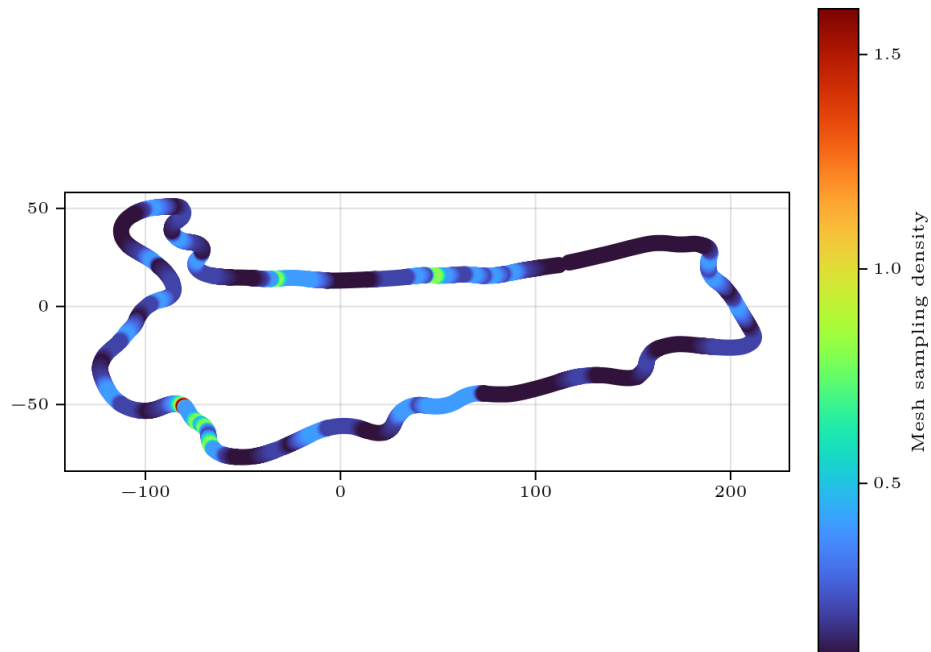


Figure 3.6: Node-interval ODE error for power-test case.

[27] to decide where to split and remove nodes.

■ 3.7.5 hp adaptive methods

hp adaptive methods are state of the art in mesh refinement. They are combination of increasing/decreasing mesh segments and increasing/decreasing order of approximating polynomial inside segments. One of the first hp-method and commonly cited is described in article [20] from University of Florida. They have produced numerous papers on hp- adaptive mesh refinement [9] [41]. Recent work on mesh refinement for functions with kinks [62]. They are the most complex for implementation and unfortunately I did not have the time to implement them.

■ 3.8 Adjoint methods

Adjoint methods allow for differentiating the solution of optimization problem. In this case it is function of time on states, controls and car parameters along the trajectory. This is very useful as it greatly simplifies sensitivity analysis. Traditional way of sensitivity analysis is to compute several runs with different vehicle parameters, but this allows to compute sensitivity of all parameters in one shot. I am using DiffOpt.jl [14], it is not exactly adjoint I don't really understand very well how it works

■ 3.8.1 quadrature rules

Chapter 4

Solvers for nonlinear programming

Once the objective function and constraints are created, the problem is passed to the NLP solver. Popular methods are Lagrange-Newton methods, these are sequential quadratic solvers and interior point solvers. These solvers are perhaps the most complex component used in this thesis, this chapter will cover them at high level, as full theory is beyond the scope. A comprehensive overview of Lagrange-Newton methods is in article[57], where the methods are broken down into key algorithmic components and unifying framework of solvers is presented. Figure4.1 shows the key algorithmic components.

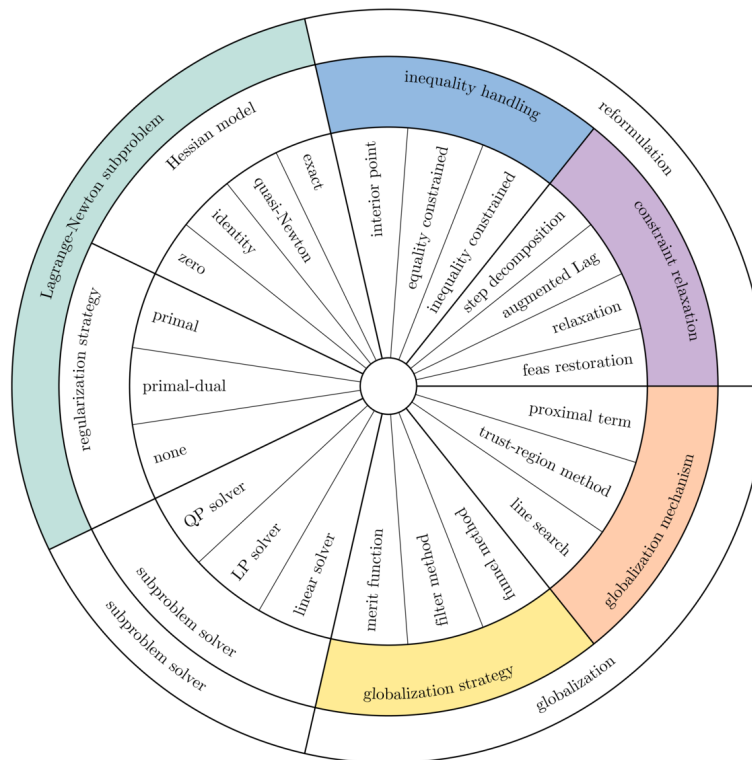


Figure 4.1: Unifying framework: the “wheel of strategies”[57]

High level overview required for this thesis of Lagrange-Newton methods is

in Figure 4.2.

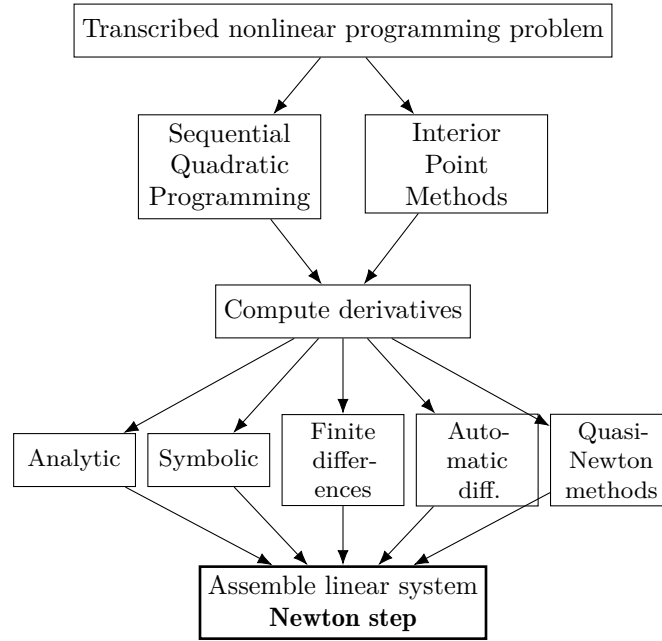


Figure 4.2: NLP solution flow: problem statement $\rightarrow$ solver family (SQP / IPM) $\rightarrow$ derivative computation $\rightarrow$ assembled Newton step.

4.1 Nonlinear programming

General NLP problems can be written as:

$$\begin{aligned}
 \min_{\mathbf{x} \in \mathbb{R}^n} \quad & f(\mathbf{x}) \\
 \text{s.t.} \quad & \mathbf{h}(\mathbf{x}) = 0 \\
 & \mathbf{g}(\mathbf{x}) \leq 0
 \end{aligned} \tag{4.1}$$

Here $\mathbf{x} \in \mathbb{R}^n$ is the decision-variable vector, $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is the scalar objective, $\mathbf{h} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ stacks the m equality constraints, and $\mathbf{g} : \mathbb{R}^n \rightarrow \mathbb{R}^p$ stacks the p inequality constraints. In the lap-time setting $\mathbf{x}$ holds the discretised states and controls, f is total lap time, $\mathbf{h}$ enforces vehicle dynamics and boundary conditions, and $\mathbf{g}$ enforces track limits, tire friction circle and actuator bounds.

4.1.1 KKT conditions

KKT conditions are the conditions of optimality, if satisfied the point is local optimum. In nonlinear programming, let's derive the KKT conditions.

$$\begin{aligned}
 \nabla f(\mathbf{x}) + \sum_{i=1}^m \lambda_i \nabla h_i(\mathbf{x}) + \sum_{i=1}^p \mu_i \nabla g_i(\mathbf{x}) &= 0 \\
 \mathbf{h}(\mathbf{x}) &= \mathbf{0} \\
 \mathbf{g}(\mathbf{x}) &\leq \mathbf{0} \\
 \mu_i &\geq 0, \quad i = 1, \dots, p \\
 \mu_i g_i(\mathbf{x}) &= 0, \quad i = 1, \dots, p
 \end{aligned} \tag{4.2}$$

4.2 Interior point methods: IPOPT, MadNLP

Interior point solvers or barrier solvers work by reformulating the NLP into a barrier problem at each step and solving a sequence of these barrier problems, with decreasing barrier parameter. Solver used in this thesis is IPOPT [63] and MadNLP[54].

4.2.1 Barrier problem formulation

There are multiple ways to transform NLP in to barrier problem, this section will briefly describe how IPOPT does it. IPOPT article [63] Describes how to transform generic NLP into form 4.3

$$\begin{aligned}
 \min_{\mathbf{x} \in \mathbb{R}^n} \quad & \phi_\mu(\mathbf{x}) := f(\mathbf{x}) - \mu \sum_{i=1}^n \ln(x_i^{(i)}) \\
 \text{s.t.} \quad & c(\mathbf{x}) = 0
 \end{aligned} \tag{4.3}$$

The Lagrangian of the barrier problem is defined as

$$\mathcal{L}(\mathbf{x}, \lambda, z) := f(\mathbf{x}) + c(\mathbf{x})^\top \lambda - z. \tag{4.4}$$

Then barrier problem is further transformed into set of linear equations, a KKT linear system, that computes the Newton step:

$$\begin{bmatrix} W_k & A_k & -I \\ A_k^\top & 0 & 0 \\ Z_k & 0 & X_k \end{bmatrix} \begin{bmatrix} d_x^k \\ d_\lambda^k \\ d_z^k \end{bmatrix} = - \begin{bmatrix} \nabla f(x_k) + A_k \lambda_k - z_k \\ c(x_k) \\ X_k Z_k e - \mu_j e \end{bmatrix}. \tag{4.5}$$

Here $W_k = \nabla^2 \mathcal{L}(x_k, \lambda_k, z_k)$ is the Hessian of the Lagrangian, A_k is the Jacobian of the equality constraints $c(x)$ and $\nabla f(x_k)$ is the gradient of the objective. The unknowns d_x^k , d_λ^k and d_z^k are the Newton steps in the primal variables, equality multipliers and bound multipliers; μ_j is the barrier parameter, which is driven to zero across outer iterations.

There are numerous commercial and free algorithms to solve this. For smaller systems it is LU decomposition, but for larger sparse problems a dedicated solver is a must. IPOPT is most commonly used with MUMPS [10] solver which is free. There are several linear commercial solvers, Harwell Subroutine Library(HSL) offers ma27 ma87 ma97 [30].Panua technologies

offers PARDISO solver [53]. Harwell Subroutine Library solvers are available free for academic usage and PARDISO solver is available free for academic usage. Latter solvers run on CPU, Nvidia has linear solver CuDSS[47] which runs on GPU. Linear solver are capable of multithreading on CPU, which needs to be explicitly turned on.

4.3 Sequential quadratic programming

Sequential quadratic programming is less commonly used for offline trajectory optimisation, usage of IPOPT prevails.

4.4 Calculation of derivatives

Major complication of many Lagrange-Newton methods is that they require gradient of the objective function, Jacobian of the constraints and Hessian of the Lagrangian to compute the Newton step. Computing these derivatives can be time intensive. The most universal method is finite differences, which means evaluating the function around the current point and estimating the derivative, this works even for a black box model, but the derivatives are not exact. When the model is available, symbolic differentiation can be used, however, the full symbolic expression is often too large to be evaluated effectively due to repeated chain and product rules. Automatic differentiation aims to solve the problems of previous methods, by applying the chain rule to the elementary operations of a function. A comprehensive review of automatic differentiation is given in [19].

4.5 Scaling and matrix conditioning

Common pitfall of nonlinear programming is poor scaling. This happens when the solver encounters variables, that are orders of magnitude apart, for example motor force in thousands of newtons and heading angle in radians. Newton steps in IPOPT are theoretically scale invariant, but the rest of the algorithm is not. [63, p. 21]. With improper scaling Lagrange multipliers in 4.5 may become large when active constraints are nearly linearly dependant and this causes numerical difficulties. TOTO SOM CELKOM OPSIAL Z IPOPT PAPERU ALE NEVIEM AKO TO INAK NAPISAT VLSTNYIM SLOVAMI.

Another thing which led to issues is constraints qualification. Linear independence constraint qualification requires gradients of the active constraints to be linearly independent. If it is violated, the Jacobian of the constraints is rank deficient and linear solve may fail. This happens when for example one constraint is created twice.

4.6 Sparsity

Sparsity is a desirable property. Sparse Hessian and Jacobian matrices contain a large fraction of zero entries, which interior-point solvers exploit through specialized linear algebra. Sparsity in the constraints themselves is also important: individual constraints should involve only a small subset of the decision variables, rather than being formulated as deeply nested nonlinear expressions. Example of this is that state derivatives of vehicle at each point, can be fully determined from only 7 variables: accelerator pedal, brake pedal, steering angle, forward speed, lateral speed, yaw rate and heading. Using only these variables results in a deeply nested nonlinear expression in the constraints. Adding redundant decision variables like forces on all wheels can reduce the iterations needed and calculation time of derivatives. Counter intuitively this larger problem was faster to solve, this is elaborated on in [33, 34]. Certain solvers can heavily exploit the sparsity structure, such as FATROP[58]. In source [32] Joris Gillis a CasADi developer compares IPOPT to FATROP, with **FATROP being 33 times faster on a trajectory optimisation problem**. Also, [15] describes how sparsity structure helps with faster derivatives approximation with finite difference and Quasi-Newton methods.

4.7 Initialization

NLP solvers require initial guess for all decision variables, to start solving. For this problem initial guess is obtained driving the vehicle on given track. PD controller is used to keep the car on the centerline and P controller is used to keep velocity at 5m/s. In general, it is possible to initialize all variables at zero, but with large scale nonlinear programming this approach is not used. For this problem initialization with zeros leads to singularity in equation 2.13 and is not possible at all.

4.8 Global and local optimums

This optimization problem is generally non-convex, therefore any local optimum does not guarantee global optimum. Commonly used NLP solvers, like IPOPT guarantee only local optimums and not global ones. Good initialization is basis for finding a local optimum close to the global optimum [39, §5.1, p. 11].



Part II

Implementation of Optimal Control problem

Chapter 5

Frameworks for Optimal Control Problem Formulation

Process of solving optimal control problem has 3 parts. Problem definition, solving of nonlinear program, optional mesh refinement

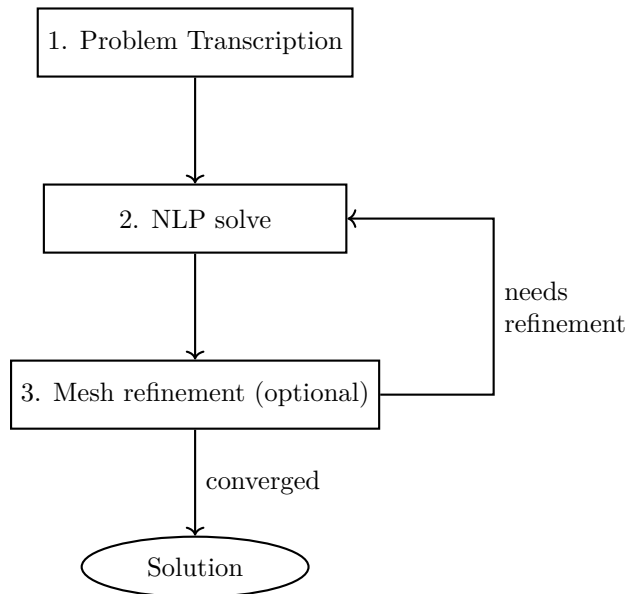


Figure 5.1: Optimal control problem solving pipeline.

The consensus in optimal control community about tools for problem formulation is that:

There is no shortage of optimal control software based on pseudospectral approaches. There are a number of other optimal control libraries that tackle similar kinds of problems, such as OTIS4, GPOPS-II, and CASADI.

[21][?] This chapter is about tool that define the OCP, and supply derivatives of the problem, they cannot solve the problem by themselves.

5.1 GPOPS-II

GPOPS-II [49] is perhaps the most advanced tool for optimal control, it was developed at University of Florida by Michael Patterson and Anil V. Rao. Other tools allow defining the problem, but GPOPS-II is a level higher. It also includes advanced mesh refinement strategies, multiple phase OCP and automatic scaling. For solving the problem it uses IPOPT or SNOPT. It interfaces with MATLAB and C++, uses ADiGator [60] for automatic differentiation. This tool is commercial and closed source. Implementation in this thesis is heavily based on work of community from University of Florida.

5.2 CasADi

CasADi is an open-source tool for nonlinear optimization and algorithmic differentiation.[17][11]. interfaces with C++,Python,MATLAB. It provides a framework INE SLOVO to build the problem, and calculate the hessian and jacobian for nlp solver it does not have any mesh refinement algorithms. Commonly paired with IPOPT for nonlinear programming, but is compatible with large variety of solvers.

5.3 dymos + OpenMDAO

Python has hp mesh refinement uses OpenMDAO [26] developed at NASA Glenn research center for evaluating aircraft concepts. Lobatto and radau collocation as default [21]. According to forum post from julia discourse, which may be biased towards julia it is slower compared to infiteOpt in julia [6]. But it has mesh refinement so that is good.

5.4 Acados

Acados is a modular open-source framework for fast embedded optimal control, developed at University of Freiburg [59]. It has C, Python, MATLAB + Simulink and Octave interface. This framework support generating C code. It is demonstrated by numerous real-world examples, such as drone racing with replanning, robot navigation in cluttered environment and many others [8].

5.5 Julia + JuMP/ infiteopt/ examodels

ExaModels, JuMP

JuMP is the go to tool for algebraic modelling in julia, it is mature, versatile. Supports Linear programming, quadratic programming, mixed linear programming, Nonlinear programming. ExaModels [56] is a specialized algebraic

modelling tool, it utilisies Single instruction, multiple data methods, enabling derivative evaluation on GPU accelerators. Coupled with MadNLP solver, the entire computation of nonlinear optimisation can be run on a GPU.

Julia options

1. Julia + jump
2. Julia + jump + infinte opt
3. julia + ExaModels

Infite opt already has various collocation methods implemented. For learning purposes and in fear that in future down the road i would realise that infniteopt blocks all the stuff i want to do i decided to use jump, without infite opt.

No julia package has mesh refinement I chose julia + jump,

Table 5.1: Comparison of Julia (JuMP + InfiniteOpt) and Python (OpenMDAO + dymos) stacks for optimal control problem formulation.

Aspect	Julia + JuMP + InfiniteOpt	Python + OpenMDAO + dymos
Language / runtime	Julia, JIT-compiled, no GIL	Python, interpreted, GIL (NumPy releases)
License	MIT (all components)	Apache 2.0
Algebraic modelling style	Macro-based (<code>@variable</code> , <code>@constraint</code> , <code>@objective</code>)	Component graph (ExplicitComponent, Group, promotes/connects)
Derivatives	AD via MathOptInterface; sparsity from expression graph	MAUD unified derivatives; mix analytic / AD / FD / complex-step per component, framework chains them
Transcription methods	Finite difference (Forward, Central, Backward); orthogonal collocation on finite elements with Gauss-Lobatto nodes only	Radau pseudospectral, Gauss-Lobatto, explicit shooting, ExplicitShooting RK
Mesh / grid refinement	None built-in; user must resolve on refined supports	Built-in ph- and hp-adaptive grid refinement
Multi-phase trajectories	Manual; user concatenate InfiniteModels and link	Native Trajectory and Phase with linkage constraints
Multidisciplinary coupling	Not supported; pure infinite-dim model	Native MDO coupling with implicit/explicit components, cyclic groups, nonlinear/linear sub-solvers
NLP solvers	IPOPT, MadNLP, Knitro, SNOPT and any MOI-compatible solver	IPOPT, SNOPT, scipy via py-OptSparse
GPU acceleration	Yes via ExaModels + MadNLP (SIMD on GPU)	Not native
Parallelism	Native multithreading, Distributed, MPI	MPI across components
Sparse total Jacobian	Yes, from MOI expression graph	Yes, MAUD assembles sparse total Jacobian from component partials
Maturity / community	JuMP mature in OR / energy; InfiniteOpt newer, smaller user base	NASA-backed, large aerospace user base
Learning curve for OCP only	Low; close to mathematical formulation	Higher; OpenMDAO component model required even for single-discipline OCP

Chapter 6

My implementation, julia + JuMP

I have decided to use julia + JuMP, because I wanted to use free and open source software.

I have already done a lot in Python and julia seemed like new interesting thing to learn. Julia is supposed to be fast and high level language, bridging the gap between C++ and python. Julia packages offer a lot of ready to use mathematical tools developed by community. Julia has

JuMP [42] (Julia for Mathematical Programming) is a modelling language and collection of optimization packages in Julia. JuMP makes it possible to create decision variables and constraints symbolically and uses automatic differentiation, to supply first and second derivative to solvers. Solver used in this thesis is IPOPT [63] and MadNLP[55][54]. MadNLP also allows utilising GPU for solver computation.

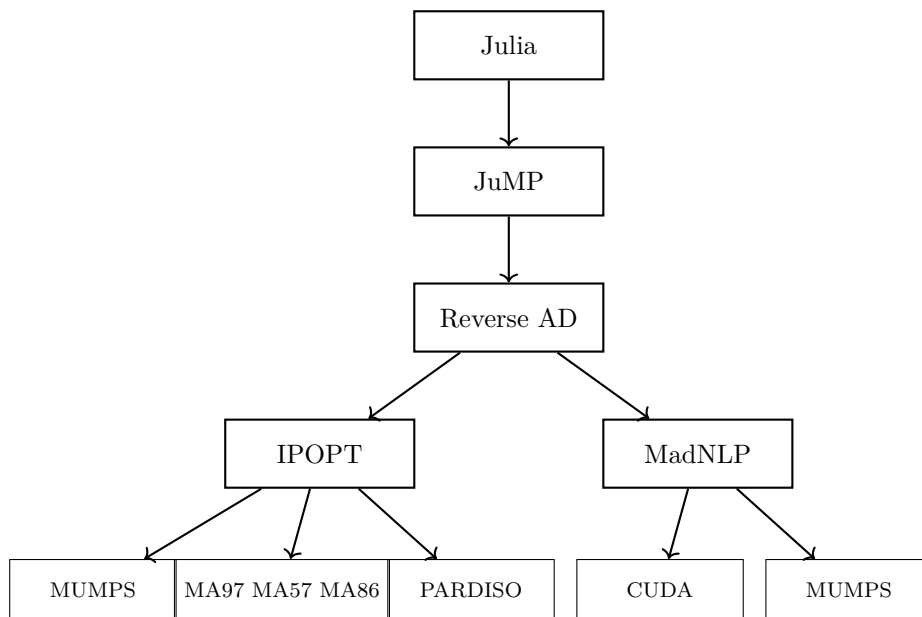


Figure 6.1: Optimization software stack

6.1 sensitivity Analysis

I have used DiffOpt.jl [14] for the sensitivity analysis. By adding vehicle parameters such as mass, lift coefficient, CoG position etc. into optimisation as decision variables fixed to a value by constraints. This adds them to the algebraic model, then the solution of minimum laptime can be differentiated by these parameters. AKO POMENOVAT TEN STROM CO JUMP VYTCORI TEN GRAF? algebraic model. Result of this is on figure 6.2

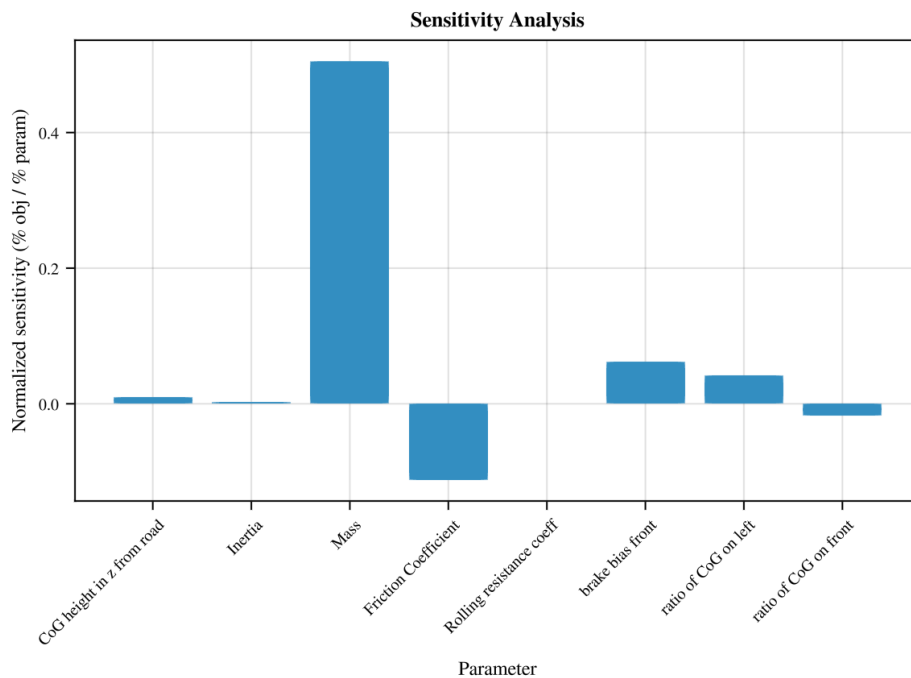


Figure 6.2: Node-interval ODE error for power-test case.

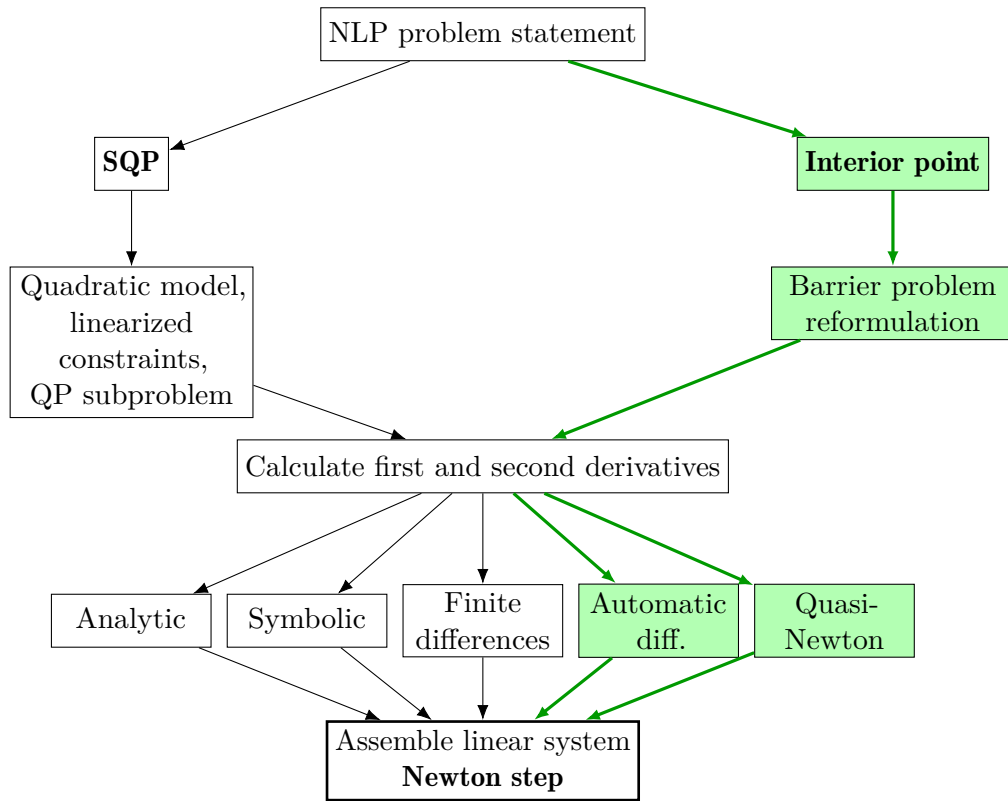


Figure 6.3: NLP solution flow with the methods used in this thesis highlighted in green: interior point with barrier reformulation (IPOPT, MadNLP), automatic differentiation provided by JuMP, and the quasi-Newton (L-BFGS) Hessian approximation used in IPOPT’s `limited-memory` mode.

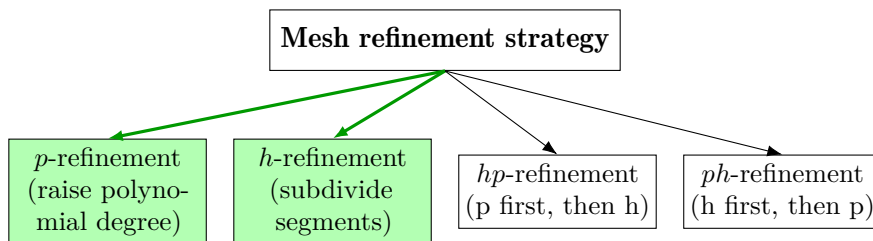


Figure 6.4: Mesh refinement layer with the strategies implemented in this thesis highlighted in green: p -refinement (raising the polynomial degree on fixed segments) and h -refinement (subdividing segments at fixed degree).

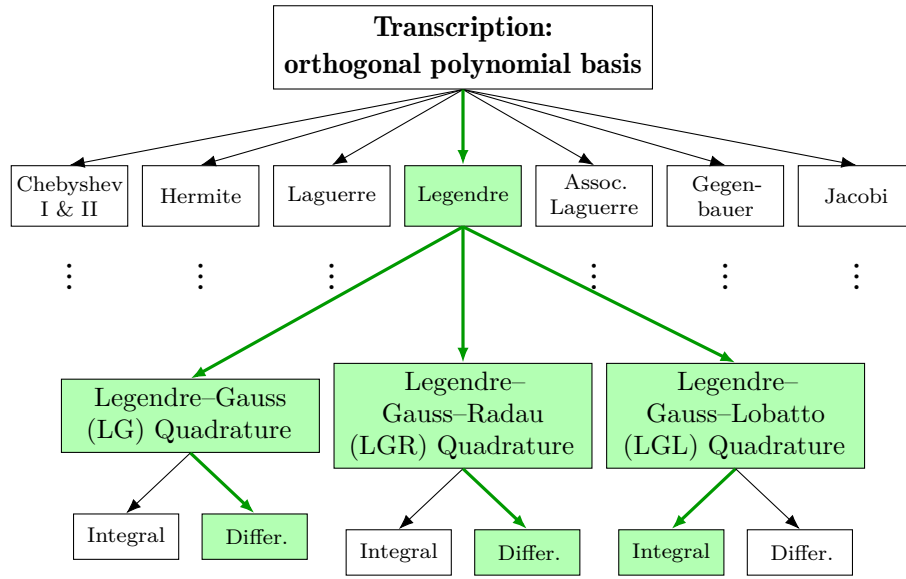


Figure 6.5: Transcription layer with the methods used in this thesis highlighted in green: Legendre basis with all three Gauss-type quadrature schemes (LG, LGR, LGL), where LG and LGR are implemented in differential form and LGL in integral form.



Part III

results

Chapter 7

comparison of methods

Performance of NLP solvers and frameworks is often tested on benchmark problems. like CUTEst [25] dataset? Tests were performed on Ryzen 9 9950X and MadNLP ran on Nvidia RTX 5070. Important caveat is that IPOPT has many (probably too many) options which can be adjusted, and that is true for all solvers. Some of differences in this benchmark may be due to sub optimal setup of the solver.

Table 7.2: Benchmark summary statistics by transcription variant.

transcription	runs	ok	fail	rate	mean_s	med_s	min_s	max_s
Legendre	14	13	1	92.8571	115.5586	25.9074	0.5774	1009.4537
Lobatto IIIA	14	13	1	92.8571	66.4536	22.3981	0.2790	532.3975
Radau	14	14	0	100.0000	71.6885	21.5821	0.4218	572.8931

Table 7.4: Benchmark results by track, vehicle model, and transcription variant.

track	car	transcription	ok	status	time_s	obj
FSCZ	singletrack	Radau	yes	solved	29.1453	50.2106
FSCZ	singletrack	Legendre	yes	solved	49.7079	50.5890
FSCZ	singletrack	Lobatto IIIA	yes	solved	28.0042	51.4710
FSCZ	twintrack	Radau	yes	solved	109.6182	57.3381
FSCZ	twintrack	Legendre	no	locally infeasible	381.2216	NaN
FSCZ	twintrack	Lobatto IIIA	no	locally infeasible	224.8871	NaN
Berlin	singletrack	Radau	yes	solved	25.9621	79.4234
Berlin	singletrack	Legendre	yes	solved	44.6565	79.4033
Berlin	singletrack	Lobatto IIIA	yes	solved	39.2681	79.1299
Berlin	twintrack	Radau	yes	solved	67.7176	77.7439
Berlin	twintrack	Legendre	yes	solved	134.5996	77.7883
Berlin	twintrack	Lobatto IIIA	yes	solved	52.1696	77.8983
Berlin	formula	Radau	yes	solved	65.8456	87.9477
Berlin	formula	Legendre	yes	solved	96.1020	88.3235
Berlin	formula	Lobatto IIIA	yes	solved	76.7914	87.9482
Berlin	bus	Radau	yes	solved	572.8931	106.9827
Berlin	bus	Legendre	yes	solved	1009.4537	106.6620
Berlin	bus	Lobatto IIIA	yes	solved	532.3975	105.8631
FSG	singletrack	Radau	yes	solved	17.2021	58.2463
FSG	singletrack	Legendre	yes	solved	25.9074	58.1609
FSG	singletrack	Lobatto IIIA	yes	solved	22.3981	56.8773
FSG	twintrack	Radau	yes	solved	86.4238	69.3149
FSG	twintrack	Legendre	yes	solved	109.9323	70.3348
FSG	twintrack	Lobatto IIIA	yes	solved	95.6528	69.8878
DoubleTurn	singletrack	Radau	yes	solved	0.4218	5.7087
DoubleTurn	singletrack	Legendre	yes	solved	0.5774	5.6794
DoubleTurn	singletrack	Lobatto IIIA	yes	solved	0.2790	6.0247
DoubleTurn	twintrack	Radau	yes	solved	2.1042	6.5283
DoubleTurn	twintrack	Legendre	yes	solved	1.2756	6.5835
DoubleTurn	twintrack	Lobatto IIIA	yes	solved	1.5196	7.0205
DoubleTurn	bus	Radau	yes	solved	2.2998	8.9075
DoubleTurn	bus	Legendre	yes	solved	1.9180	8.9864
DoubleTurn	bus	Lobatto IIIA	yes	solved	1.8477	8.8676
FigureEight	singletrack	Radau	yes	solved	1.5903	14.9127
FigureEight	singletrack	Legendre	yes	solved	1.9768	14.9162
FigureEight	singletrack	Lobatto IIIA	yes	solved	1.3477	14.8296
FigureEight	twintrack	Radau	yes	solved	12.5541	16.5690
FigureEight	twintrack	Legendre	yes	solved	20.7391	16.5736
FigureEight	twintrack	Lobatto IIIA	yes	solved	9.2105	16.5319
FigureEight	bus	Radau	yes	solved	9.8610	22.8420
FigureEight	bus	Legendre	yes	solved	5.4157	22.7966
FigureEight	bus	Lobatto IIIA	yes	solved	3.0111	22.3442

Table 7.6: Linear solver benchmark summary statistics.

linear solver	runs	ok	fail	rate	mean_s	med_s	min_s	max_s
ma27	14	14	0	100.0000	97.1562	12.1841	0.2777	964.2969
ma57	14	14	0	100.0000	93.3249	12.6211	0.2897	895.8892
ma97	14	14	0	100.0000	107.8802	12.1752	0.2689	1099.4816
mumps	14	14	0	100.0000	69.6295	12.6470	0.2803	569.8588

Table 7.8: Linear solver benchmark results by track, vehicle model, and transcription variant.

track	car	linear solver	ok	status	time_s	obj
FSCZ	singletrack	mumps	yes	solved	11.4861	50.2106
FSCZ	singletrack	ma57	yes	solved	11.2823	50.2106
FSCZ	singletrack	ma27	yes	solved	11.2823	50.2106
FSCZ	singletrack	ma97	yes	solved	11.2701	50.2106
FSCZ	twintrack	mumps	yes	solved	108.9638	57.3381
FSCZ	twintrack	ma57	yes	solved	91.0531	57.3381
FSCZ	twintrack	ma27	yes	solved	108.9439	57.2790
FSCZ	twintrack	ma97	yes	solved	92.4024	57.3381
Berlin	singletrack	mumps	yes	solved	26.6890	79.4234
Berlin	singletrack	ma57	yes	solved	24.6851	79.4234
Berlin	singletrack	ma27	yes	solved	25.5493	79.4234
Berlin	singletrack	ma97	yes	solved	25.6775	79.4234
Berlin	twintrack	mumps	yes	solved	67.2587	77.7439
Berlin	twintrack	ma57	yes	solved	58.9130	77.7439
Berlin	twintrack	ma27	yes	solved	51.7979	77.7439
Berlin	twintrack	ma97	yes	solved	55.8040	77.7439
Berlin	formula	mumps	yes	solved	64.5359	87.9477
Berlin	formula	ma57	yes	solved	62.7170	87.9477
Berlin	formula	ma27	yes	solved	60.3951	87.9477
Berlin	formula	ma97	yes	solved	73.0484	87.9477
Berlin	bus	mumps	yes	solved	569.8588	106.9827
Berlin	bus	ma57	yes	solved	895.8892	106.9795
Berlin	bus	ma27	yes	solved	964.2969	106.9815
Berlin	bus	ma97	yes	solved	1099.4816	106.6410
FSG	singletrack	mumps	yes	solved	12.7246	58.2463
FSG	singletrack	ma57	yes	solved	13.9599	58.2463
FSG	singletrack	ma27	yes	solved	13.0858	58.2463
FSG	singletrack	ma97	yes	solved	13.0492	58.2463
FSG	twintrack	mumps	yes	solved	85.2451	69.3149
FSG	twintrack	ma57	yes	solved	123.3800	69.3471
FSG	twintrack	ma27	yes	solved	100.8860	69.3140
FSG	twintrack	ma97	yes	solved	115.3881	69.3140
DoubleTurn	singletrack	mumps	yes	solved	0.2803	5.7087
DoubleTurn	singletrack	ma57	yes	solved	0.2897	5.7087
DoubleTurn	singletrack	ma27	yes	solved	0.2777	5.7087
DoubleTurn	singletrack	ma97	yes	solved	0.2689	5.7087
DoubleTurn	twintrack	mumps	yes	solved	2.0312	6.5283
DoubleTurn	twintrack	ma57	yes	solved	2.8587	6.5283
DoubleTurn	twintrack	ma27	yes	solved	1.8017	6.5283
DoubleTurn	twintrack	ma97	yes	solved	1.9207	6.5283
DoubleTurn	bus	mumps	yes	solved	2.2462	8.9075
DoubleTurn	bus	ma57	yes	solved	1.5365	8.9075
DoubleTurn	bus	ma27	yes	solved	1.3301	8.9075
DoubleTurn	bus	ma97	yes	solved	1.3733	8.9075
FigureEight	singletrack	mumps	yes	solved	0.8838	14.9127
FigureEight	singletrack	ma57	yes	solved	0.8841	14.9127
FigureEight	singletrack	ma27	yes	solved	0.9921	14.9127
FigureEight	singletrack	ma97	yes	solved	0.9997	14.9127
FigureEight	twintrack	mumps	yes	solved	12.5693	16.5690
FigureEight	twintrack	ma57	yes	solved	11.0038	16.5690
FigureEight	twintrack	ma27	yes	solved	11.1527	16.5690
FigureEight	twintrack	ma97	yes	solved	11.3013	16.5690
FigureEight	bus	mumps	yes	solved	10.0401	22.8420
FigureEight	bus	ma57	yes	solved	8.0956	22.8420
FigureEight	bus	ma27	yes	solved	8.3957	22.8420
FigureEight	bus	ma97	yes	solved	8.3372	22.8420

Table 7.10: NLP solver / backend benchmark summary statistics (IPOPT-MUMPS, MadNLP-CPU, MadNLP-GPU).

var <i>String</i>	runs <i>Int64</i>	ok <i>Int64</i>	fail <i>Int64</i>	rate <i>Float64</i>	mean_s <i>Float64</i>	med_s <i>Float64</i>	min_s <i>Float64</i>	max_s <i>Float64</i>
ipopt-mumps	14	14	0	100.0000	120.7494	11.4284	0.2772	1286.3330
madnlp-cpu	14	14	0	100.0000	120.0697	13.4490	0.2746	1312.4952
madnlp-gpu	14	14	0	100.0000	58.6134	18.3702	0.3292	395.6836

Table 7.12: NLP solver / backend benchmark results by track, vehicle model, and transcription variant.

track <i>String</i>	car <i>String</i>	var <i>String</i>	ok <i>String</i>	status <i>String</i>	time_s <i>Float64</i>	obj <i>Float64</i>
FSCZ	singletrack	ipopt-mumps	yes	solved	10.1717	50.2106
FSCZ	singletrack	madnlp-cpu	yes	solved	11.0951	50.2106
FSCZ	singletrack	madnlp-gpu	yes	solved	13.1153	50.2098
FSCZ	twintrack	ipopt-mumps	yes	solved	102.9340	57.3381
FSCZ	twintrack	madnlp-cpu	yes	solved	96.9121	57.2490
FSCZ	twintrack	madnlp-gpu	yes	solved	80.8375	57.2490
Berlin	singletrack	ipopt-mumps	yes	solved	25.3289	79.4234
Berlin	singletrack	madnlp-cpu	yes	solved	17.0487	79.4234
Berlin	singletrack	madnlp-gpu	yes	solved	23.9279	79.4214
Berlin	twintrack	ipopt-mumps	yes	solved	85.5972	77.7417
Berlin	twintrack	madnlp-cpu	yes	solved	56.3039	77.7442
Berlin	twintrack	madnlp-gpu	yes	solved	44.1605	77.7421
Berlin	formula	ipopt-mumps	yes	solved	62.9868	87.9477
Berlin	formula	madnlp-cpu	yes	solved	59.3588	87.9477
Berlin	formula	madnlp-gpu	yes	almost locally solved	115.1056	87.9213
Berlin	bus	ipopt-mumps	yes	solved	1286.3330	107.2120
Berlin	bus	madnlp-cpu	yes	solved	1312.4952	106.6728
Berlin	bus	madnlp-gpu	yes	solved	395.6836	106.8320
FSG	singletrack	ipopt-mumps	yes	solved	12.1801	58.2463
FSG	singletrack	madnlp-cpu	yes	solved	12.3199	58.2463
FSG	singletrack	madnlp-gpu	yes	solved	12.3258	58.2455
FSG	twintrack	ipopt-mumps	yes	solved	80.6315	69.3149
FSG	twintrack	madnlp-cpu	yes	solved	86.6410	69.4039
FSG	twintrack	madnlp-gpu	yes	solved	95.2812	69.3894
DoubleTurn	singletrack	ipopt-mumps	yes	solved	0.2772	5.7087
DoubleTurn	singletrack	madnlp-cpu	yes	solved	0.2746	5.7087
DoubleTurn	singletrack	madnlp-gpu	yes	solved	0.3292	5.7086
DoubleTurn	twintrack	ipopt-mumps	yes	solved	1.9573	6.5283
DoubleTurn	twintrack	madnlp-cpu	yes	solved	3.4973	6.5463
DoubleTurn	twintrack	madnlp-gpu	yes	solved	2.3026	6.5282
DoubleTurn	bus	ipopt-mumps	yes	solved	1.2222	8.9075
DoubleTurn	bus	madnlp-cpu	yes	solved	1.2294	8.9075
DoubleTurn	bus	madnlp-gpu	yes	solved	1.5347	8.9074
FigureEight	singletrack	ipopt-mumps	yes	solved	0.8630	14.9127
FigureEight	singletrack	madnlp-cpu	yes	solved	0.9217	14.9127
FigureEight	singletrack	madnlp-gpu	yes	solved	1.1726	14.9124
FigureEight	twintrack	ipopt-mumps	yes	solved	10.6767	16.5690
FigureEight	twintrack	madnlp-cpu	yes	solved	14.5781	16.5658
FigureEight	twintrack	madnlp-gpu	yes	solved	23.6250	16.5987
FigureEight	bus	ipopt-mumps	yes	solved	9.3313	22.8420
FigureEight	bus	madnlp-cpu	yes	solved	8.2996	22.7936
FigureEight	bus	madnlp-gpu	yes	solved	11.1854	22.7984

Table 7.14: Hessian approximation benchmark summary statistics (exact vs. limited-memory quasi-Newton).

var <i>String</i>	runs <i>Int64</i>	ok <i>Int64</i>	fail <i>Int64</i>	rate <i>Float64</i>	mean_s <i>Float64</i>	med_s <i>Float64</i>	min_s <i>Float64</i>	max_s <i>Float64</i>
ipopt-exact	14	14	0	100.0000	120.7661	11.9731	0.2891	1283.6471
ipopt-lbfgs	14	7	7	50.0000	12.0994	13.5710	0.2880	27.7549

Table 7.16: Hessian approximation benchmark results by track, vehicle model, and transcription variant.

track	car	var	ok	status	time_s	obj
<i>String</i>	<i>String</i>	<i>String</i>	<i>String</i>	<i>String</i>	<i>Float64</i>	<i>Float64</i>
FSCZ	singletrack	ipopt-exact	yes	solved	11.4260	50.2106
FSCZ	singletrack	ipopt-lbfgs	yes	solved	13.5710	50.2106
FSCZ	twintrack	ipopt-exact	yes	solved	101.6519	57.3381
FSCZ	twintrack	ipopt-lbfgs	no	iteration limit	724.0525	NaN
Berlin	singletrack	ipopt-exact	yes	solved	25.6411	79.4234
Berlin	singletrack	ipopt-lbfgs	yes	solved	27.7549	79.4234
Berlin	twintrack	ipopt-exact	yes	solved	83.5702	77.7417
Berlin	twintrack	ipopt-lbfgs	no	iteration limit	2383.7898	NaN
Berlin	formula	ipopt-exact	yes	solved	65.2719	87.9477
Berlin	formula	ipopt-lbfgs	no	iteration limit	754.0812	NaN
Berlin	bus	ipopt-exact	yes	solved	1283.6471	107.2120
Berlin	bus	ipopt-lbfgs	no	iteration limit	654.4284	NaN
FSG	singletrack	ipopt-exact	yes	solved	12.1035	58.2463
FSG	singletrack	ipopt-lbfgs	yes	solved	14.5613	58.2463
FSG	twintrack	ipopt-exact	yes	solved	80.4082	69.3149
FSG	twintrack	ipopt-lbfgs	no	iteration limit	1274.5974	NaN
DoubleTurn	singletrack	ipopt-exact	yes	solved	0.2891	5.7087
DoubleTurn	singletrack	ipopt-lbfgs	yes	solved	0.2880	5.7087
DoubleTurn	twintrack	ipopt-exact	yes	solved	1.9316	6.5283
DoubleTurn	twintrack	ipopt-lbfgs	yes	almost locally solved	19.2166	6.5412
DoubleTurn	bus	ipopt-exact	yes	solved	2.4195	8.9075
DoubleTurn	bus	ipopt-lbfgs	yes	solved	7.2745	8.9075
FigureEight	singletrack	ipopt-exact	yes	solved	0.9486	14.9127
FigureEight	singletrack	ipopt-lbfgs	yes	solved	2.0296	14.9127
FigureEight	twintrack	ipopt-exact	yes	solved	11.8428	16.5690
FigureEight	twintrack	ipopt-lbfgs	no	iteration limit	481.8881	NaN
FigureEight	bus	ipopt-exact	yes	solved	9.5744	22.8420
FigureEight	bus	ipopt-lbfgs	no	iteration limit	67.0283	NaN



Chapter 8

Conclusion

All goals were achieved. One thing which was not a goal but, would be super useful if i managed to do a validation on real world data. Validation of sensitivity analysis. I wanted to do it for a long time but there was not enough of it.

Julia was interesting language to learn, it offers blazing fast execution, but that comes with caveats. Debugging is way more complicated. When working on a smaller project, the debugging is not such a pain, but when the project is bigger debugging is not user friendly. Debugging in python and MATLAB is way easier. Julias selling point is that it is just in time compiled language, rivalling C. That is true BUT you have to know what you are doing, if you dont declare types in functions it is as slow as python.

At the begging of this project I wanted a more complex model, I wanted to go deeper with complexity than my bachelor thesis. That is add pitch, roll and full model of suspension into the simulation. Also this simulation does not account for gearboxes with discrete stages, and no combustion engine.

Believing 100% this tool and designing car according to it would most likely lead to undriveable vehicle, as there is no force pushing for stability driveability in NLP [50] deals also with this. The optimal driving created by NLP is unachievable by real driver, as the vehicle is on its limit and one mistake woul overload tires and cause failure.

Track model is also heavily simplified, neglecting slope and bank of the track.



Appendices

Appendix A

Bibliography

- [1] 2026 Class Project: Race Car Minimum Lap Time - AE6310 Course Notes. https://sm-dogroup.github.io/ae6310/jupyter/2026_project/race_car_problem.html.
- [2] A Computationally Efficient Framework for Free-trajectory Minimum-lap-time Optimization of Racing Cars. <https://arxiv.org/html/2511.13522v1>.
- [3] OptimumLap | OptimumG.
- [4] Solving ordinary differential equations I. nonstiff problems. *Mathematics and Computers in Simulation*, 29(5):447, October 1987.
- [5] Solving ordinary differential equations I. nonstiff problems. *Mathematics and Computers in Simulation*, 29(5):447, October 1987.
- [6] Does anybody know OpenMDAO - Specific Domains / Optimization (Mathematical). <https://discourse.julialang.org/t/does-anybody-know-openmdao/122840>, November 2024.
- [7] Coin-or/CppAD. COIN-OR Foundation, May 2026.
- [8] acados Developers. acados documentation: real-world examples. https://docs.acados.org/real_world_examples/index.html, 2026. Accessed: 2026-05-14.
- [9] Yunus Agamawi and Anil Rao. CGPOPS: A C++ Software for Solving Multiple-Phase Optimal Control Problems Using Adaptive Gaussian Quadrature Collocation and Sparse Nonlinear Programming. *ACM Transactions on Mathematical Software*, 46, May 2020.
- [10] P.R. Amestoy, I. S. Duff, J. Koster, and J.-Y. L’Excellent. A fully asynchronous multifrontal solver using distributed dynamic scheduling. *SIAM Journal on Matrix Analysis and Applications*, 23(1):15–41, 2001.
- [11] Joel A E Andersson, Joris Gillis, Greg Horn, James B Rawlings, and Moritz Diehl. CasADi – A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1):1–36, 2019.

- [12] Uri M. Ascher, Robert M. M. Mattheij, and Robert D. Russell. *Numerical Solution of Boundary Value Problems for Ordinary Differential Equations*. Classics in Applied Mathematics. SIAM, 1995.
- [13] Jean-Paul Berrut and Lloyd N. Trefethen. Barycentric Lagrange Interpolation. *SIAM Review*, 46(3):501–517, January 2004.
- [14] Mathieu Besançon, Joaquim Dias Garcia, Benoît Legat, and Akshay Sharma. Flexible differentiable optimization via model transformations. *INFORMS Journal on Computing*, 36(2):456–478, 2023.
- [15] John T. Betts. *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*. Society for Industrial and Applied Mathematics, second edition, January 2010.
- [16] Forbes T. Brown. *Engineering System Dynamics: A Unified Graph-Centered Approach, Second Edition*. CRC Press, Boca Raton, 2 edition, August 2006.
- [17] CasADi Developers. CasADi – an open-source tool for nonlinear optimization and algorithmic differentiation. <https://web.casadi.org/>, 2026. Accessed: 2026-05-14.
- [18] Richard Ciglanský. Analysis of vehicle components and their impact on lap-time.
- [19] Daniel Cortild, J Haastert, A Sanabria, and F Voronine. *A Brief Review of Automatic Differentiation*. March 2023.
- [20] Christopher L. Darby, William W. Hager, and Anil V. Rao. An hp -adaptive pseudospectral method for solving optimal control problems. *Optimal Control Applications and Methods*, 32(4):476–502, July 2011.
- [21] Robert Falck, Justin S. Gray, Kaushik Ponnappalli, and Ted Wright. *dy-mos: A python package for optimal control of multidisciplinary systems*. *Journal of Open Source Software*, 6(59):2809, 2021.
- [22] Liang Fang, Stefan Vandewalle, and Johan Meyers. A parallel-in-time multiple shooting algorithm for large-scale PDE-constrained optimal control problems. *Journal of Computational Physics*, 452:110926, 2022.
- [23] Divya Garg, Michael Patterson, William Hager, Anil Rao, David R Benson, and Geoffrey T Huntington. An overview of three pseudospectral methods for the numerical solution of optimal control problems.
- [24] Divya Garg, Michael Patterson, William W. Hager, Anil V. Rao, David A. Benson, and Geoffrey T. Huntington. A unified framework for the numerical solution of optimal control problems using pseudospectral methods. *Automatica*, 46(11):1843–1851, November 2010.

- [25] Nick Gould, Dominique Orban, and Philippe Toint. CUTEst: A Constrained and Unconstrained Testing Environment with safe threads for Mathematical Optimization. *Computational Optimization and Applications*, 60, July 2014.
- [26] Justin S. Gray, John T. Hwang, Joaquim R. R. A. Martins, Kenneth T. Moore, and Bret A. Naylor. OpenMDAO: An open-source framework for multidisciplinary design, analysis, and optimization. *Structural and Multidisciplinary Optimization*, 59(4):1075–1104, April 2019.
- [27] A. Harten, B. Engquist, S. Osher, and S. R. Chakravarthy. Uniformly high order accurate essentially non-oscillatory schemes, III. *Journal of Computational Physics*, 131(1):3–47, 1997.
- [28] Himanshu Hatwal and E. C. Mikulcik. Some inverse solutions to an automobile path-tracking problem with input control of steering and brakes. *Vehicle System Dynamics*, 15:61–71, 1986.
- [29] Alexander Heilmeyer, Maximilian Geisslinger, and Johannes Betz. A quasi-steady-state lap time simulation for electrified race cars. In *2019 Fourteenth International Conference on Ecological Vehicles and Renewable Energies (EVER)*. IEEE, May 2019.
- [30] HSL. A collection of Fortran codes for large scale scientific computation. <http://www.hsl.rl.ac.uk/>, 2011.
- [31] Shuang Li* Jisong Zhao. Adaptive mesh refinement method for solving optimal control problems using interpolation error analysis and improved data compression.
- [32] Joris Gillis. IPOPT on steroids: Fatrop solver in CasADi, June 2024.
- [33] Julia Discourse. 90% of IPOPT run time is spent in Hessian-constraints evaluation, very little time in linear solver — is it normal? <https://discourse.julialang.org/t/90-of-ipopt-run-time-is-spent-in-hessian-constraints-evaluation-very-little-ti/136708>, 2025. Accessed: 2026-04-19.
- [34] JuMP Developers. Performance problem with deeply nested nonlinear expressions. <https://github.com/jump-dev/JuMP.jl/issues/4024>, 2026. GitHub issue #4024, Accessed: 2026-05-13.
- [35] Matthew Kelly. Matthew kelly — personal website. <https://www.matthwepeterkelly.com/index.html>. Accessed: 2026-05-15.
- [36] Matthew Kelly. Trajectory optimization: cannon example. <https://www.matthwepeterkelly.com/tutorials/trajectoryOptimization/cannon.html>. Accessed: 2026-05-15.
- [37] Matthew Kelly. Introduction to trajectory optimization. YouTube, <https://www.youtube.com/watch?v=wlkRYMVUZTs>, 2017. Accessed: 2026-05-15.

- [38] Matthew Kelly. An introduction to trajectory optimization: How to do your own direct collocation. *SIAM Review*, 59(4):849–904, 2017.
- [39] Matthew Kelly. An Introduction to Trajectory Optimization: How to Do Your Own Direct Collocation. *SIAM Review*, 59(4):849–904, January 2017.
- [40] Matthew Kelly. Trajectory optimization terminology. <https://www.matthewpeterkelly.com/tutorials/trajectoryOptimization/terminology.html>, 2026. Accessed: 2026-05-14.
- [41] Fengjin Liu, William W. Hager, and Anil V. Rao. Adaptive Mesh Refinement Method for Optimal Control Using Decay Rates of Legendre Polynomial Coefficients. *IEEE Transactions on Control Systems Technology*, 26(4):1475–1483, July 2018.
- [42] Miles Lubin, Oscar Dowson, Joaquim Dias Garcia, Joey Huchette, Benoît Legat, and Juan Pablo Vielma. JuMP 1.0: Recent improvements to a modeling language for mathematical optimization. *Mathematical Programming Computation*, 2023.
- [43] Juan Manzanero. Juanmanzanero/fastest-lap, April 2026.
- [44] Matteo Massaro and David Limebeer. Minimum-lap-time optimisation and simulation. *Vehicle System Dynamics*, 59:1–45, April 2021.
- [45] mc12027. Mc12027/OpenLAP-Lap-Time-Simulator, April 2026.
- [46] Yukio Nakajima. Advanced tire mechanics. 2019.
- [47] NVIDIA Corporation. cuDSS: NVIDIA CUDA Direct Sparse Solver Library Documentation. <https://docs.nvidia.com/cuda/cudss/>, 2026. Accessed: 2026-05-12.
- [48] Michael A. Patterson, William W. Hager, and Anil V. Rao. A ph mesh refinement method for optimal control. *Optimal Control Applications and Methods*, 36(4):398–421, 2015.
- [49] Michael A. Patterson and Anil V. Rao. GPOPS-II: A MATLAB Software for Solving Multiple-Phase Optimal Control Problems Using hp-Adaptive Gaussian Quadrature Collocation Methods and Sparse Nonlinear Programming. *ACM Transactions on Mathematical Software*, 41(1):1–37, October 2014.
- [50] Giacomo Perantoni and David J.N. Limebeer. Optimal control for a Formula One car with variable parameters. *Vehicle System Dynamics*, 52(5):653–678, May 2014.
- [51] Anil Rao. A Survey of Numerical Methods for Optimal Control. *Advances in the Astronautical Sciences*, 135, January 2010.

- [52] R. D. Roland and C. F. Thelin. Computer simulation of Watkins Glen grand prix circuit performance. Technical Report ZL-5002-K-1, Cornell Aeronautical Laboratory, 1971.
- [53] Olaf Schenk, Klaus Gärtner, Wolfgang Fichtner, and A.D. Stricker. Pardiso: A high-performance serial and parallel sparse linear solver in semiconductor device simulation. *Future Generation Computer Systems*, 18:69–78, 03 2000.
- [54] Sungho Shin, Mihai Anitescu, and François Pacaud. Accelerating optimal power flow with GPUs: SIMD abstraction of nonlinear programs and condensed-space interior-point methods. *Electric Power Systems Research*, 236:110651, 2024.
- [55] Sungho Shin, Carleton Coffrin, Kaarthik Sundar, and Victor M Zavala. Graph-based modeling and decomposition of energy infrastructures. *IFAC-PapersOnLine*, 54(3):693–698, 2021.
- [56] Sungho Shin, François Pacaud, and Mihai Anitescu. Accelerating optimal power flow with GPUs: SIMD abstraction of nonlinear programs and condensed-space interior-point methods, 2023.
- [57] Charlie Vanaret and Sven Leyffer. *Implementing a unified solver for nonlinearly constrained optimization*. November 2025.
- [58] Lander Vanroye, Ajay Sathya, Joris De Schutter, and Wilm Decré. FATROP : A Fast Constrained Optimal Control Problem Solver for Robot Trajectory Optimization and Control, July 2023.
- [59] Robin Verschueren, Gianluca Frison, Dimitris Kouzoupis, Jonathan Frey, Niels van Duijkeren, Andrea Zanelli, Branimir Novoselnik, Thivaharan Albin, Rien Quirynen, and Moritz Diehl. acados – a modular open-source framework for fast embedded optimal control. *Mathematical Programming Computation*, 2021.
- [60] Matthew J Weinstein and Anil V Rao. Algorithm 984: Adigator, a toolbox for the algorithmic differentiation of mathematical functions in matlab using source transformation via operator overloading. *ACM Transactions on Mathematical Software (TOMS)*, 44(2):21, 2017.
- [61] Matthew E. Wilhelm and Matthew D. Stuber. EAGO.jl: easy advanced global optimization in Julia. *Optimization Methods and Software*, 37(2):425–450, 2022.
- [62] Hendrik Wilka and Jens Lang. Adaptive hp-Polynomial Based Sparse Grid Collocation Algorithms for Piecewise Smooth Functions with Kinks, May 2025.
- [63] Andreas Wächter and Lorenz T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, March 2006.